

UNIVERSITY OF DHAKA

**ARGUS: An Adaptive,
Resource-Aware, Grounded
Understanding System
for Real-Time Explainable Video
Anomaly Detection**

by

Exam Roll: CURZON-2323816

Registration No: 2021-711-196, Session: 2021-22

Exam Roll: CURZON-2323826

Registration No: 2021-411-180, Session: 2021-22



Bachelor of Science

Declaration Form



University of Dhaka

We hereby declare that the work reported in the thesis titled *ARGUS: An Adaptive, Resource-Aware, Grounded Understanding System for Real-Time Explainable Video Anomaly Detection* is the result of our own investigation, carried out under the supervision of Md. Ahasanul Alam, Department of Computer Science and Engineering, University of Dhaka. It has not been submitted, in whole or in part, for any academic award, here or elsewhere.

Material drawn from the work of others has been acknowledged in the text and listed in the bibliography, and every method adopted from prior work is credited where it is used. As far as we are aware, it contains no other material that has been published or written by someone else.

Md. Shahriar Kabir
Candidate 1

Tawyabul Islam Tamim
Candidate 2

Md. Ahasanul Alam
Lecturer
Department of Computer Science and Engineering
University of Dhaka
Supervisor

“I asked the detector why it had raised the alarm. It said: anomaly. I asked again, more politely.”

An operator, on the limits of the unexplained alert

UNIVERSITY OF DHAKA

Abstract

Video anomaly detection for surveillance faces three demands that no existing system satisfies at once: running continuously on limited resources, matching the accuracy of offline models that see the whole video, and producing explanations that can be checked rather than taken on trust. Recent research combines a cheap detector with an expensive vision-language model, but that cascade acts only as a switch. A threshold decides when the large model runs, the alert waits for it to return, and nothing limits what the system spends on an unfamiliar stream. We propose ARGUS, which treats the cascade as a scheduler under a stated compute budget and latency contract. The scheduler is built from three mechanisms. First, a two-deadline output contract splits the response into a calibrated probability due on a hard deadline and an explanation due on a soft one, so the time to an alert is set by the cheap detector, never by the large model, the network, or the queue. Second, a budgeted escalation controller prices access to the expensive stage and adjusts that price online. Spending is then set by the hardware, not by a threshold tuned on a validation set. One shared price also divides a single accelerator among several cameras. Third, speculative escalation, a per-camera prompt-prefix cache, and memoization of repeated evidence cut how long the expensive stage takes, not just how often it runs. The expensive stage is grounded, and its vision-language model is frozen, never trained on a set of example explanations. It must cite evidence gathered by a separate detector and tracker, so a program can check every claim. We derive a cost and latency model for the cascade and define an evaluation protocol that reports accuracy, latency, calibration and explanation faithfulness together.

Acknowledgements

We are grateful to our supervisor, Md. Ahasanul Alam, for his guidance throughout this work. His questions repeatedly sent us back to assumptions we had been treating as settled, and the shape of this thesis owes much to those returns. He gave us the freedom to pursue a formulation of the problem that was not the obvious one, and the discipline to state its limits plainly.

We owe thanks to the faculty of the Department of Computer Science and Engineering, University of Dhaka, whose teaching made this work possible and whose facilities it was carried out on. We are also grateful to our fellow students, whose willingness to hear an argument out and then find the hole in it improved this work more than they may realize.

Many of the tools and datasets this work depends on were made public by research groups under no obligation to share them. We are indebted to everyone who does so. Without that practice, a project of this scope would not have been feasible within the resources available to us.

Finally, we thank our families, for their patience during the months this work demanded and for their support long before it began.

Contents

Declaration Form	i
Abstract	iii
Acknowledgements	iv
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Motivations	1
1.2 Objectives	7
1.3 Contributions	8
1.4 Challenges	10
1.5 Organization	11
2 Related Works	12
2.1 Video Anomaly Detection	12
2.2 Traditional VAD	13
2.3 Deep Learning-based VAD	14
2.4 Zero-shot and Training-free VAD	15
2.5 Vision-Language Models for VAD	16
2.6 Existing Relevant Methods	18
2.7 Comparison of Existing Approaches	20
2.8 Research Gap	22
2.9 Summary	24
3 Proposed Methodologies	25
3.1 Problem Formulation	25
3.1.1 Operating setting	25
3.1.2 Supervision available	26
3.1.3 Objective	27
3.1.4 Notation	28
3.2 System Overview	30
3.2.1 Operation on unseen kinds of anomaly	33

3.3	The Sentinel	34
3.3.1	Architecture	34
3.3.2	The frame cache	35
3.3.3	Training objective	36
3.3.4	Evidential uncertainty	37
3.4	The Escalation Controller	38
3.4.1	Change-point statistic and escalation utility	39
3.4.2	Escalation as a constrained decision problem	40
3.4.3	Coupling the price to the queue	43
3.4.4	Speculative escalation	44
3.4.5	Adaptive window length	45
3.4.6	Arbitrating one accelerator across many cameras	46
3.4.7	Setting the controller’s constants	47
3.5	Evidence Grounder	49
3.5.1	Extraction	49
3.5.2	Structured evidence list	51
3.5.3	Canonical form	52
3.6	Scene Memory	52
3.6.1	Construction and retrieval	52
3.6.2	Updating without gradients	53
3.6.3	Bounding the memory and its errors	55
3.7	Reasoner	55
3.7.1	Why the model is frozen	55
3.7.2	Prompt construction	56
3.7.3	Contractual output and the citation check	56
3.8	Fusion and the Two-Deadline Contract	58
3.8.1	The two deadlines	58
3.8.2	Emission and amendment	59
3.8.3	Calibration as a primary metric	61
3.8.4	Failure containment	61
3.9	Cost and Latency Analysis	62
3.9.1	Cost model	62
3.9.2	Latency model	64
3.9.3	Per-camera prefix cache	64
3.9.4	Evidence-hash memoization	65
3.9.5	Deadline-aware degradation	66
3.10	The Complete Procedure	66
3.11	Summary	68
4	Implementation	69
4.1	Component Choices and Compute Budget	69
4.2	Configuration	70
4.3	Development Plan	71
4.4	Risks and Mitigations	74

5	Experimental Results	75
5.1	Evaluation Protocol	75
5.1.1	Datasets	75
5.1.2	Metrics	76
5.1.3	Baselines and reporting discipline	77
5.1.4	Ablations	78
5.2	Results Analysis	79
5.3	Summary of the Experimental Results	79
6	Conclusions	80
6.1	Research Summary	80
6.2	Deployment and Ethical Considerations	80
6.3	Future Work	80
	Bibliography	81

List of Figures

1.1	Imbalanced distribution of normal and abnormal segments in the UCF-Crime dataset	2
1.2	Conceptual illustration of video anomaly detection	2
3.1	The ARGUS pipeline	32

List of Tables

2.1	Capability matrix (papers ordered by relevance)	21
3.1	Symbols of the proposed method.	28
4.1	Initial configuration. All values are starting points to be tuned on a held-out validation split; none is a reported result.	70
4.2	Approximate fourteen-week schedule. Milestone numbers refer to the list above.	73

List of Algorithms

1	ARGUS online processing of one segment	67
---	--	----

AP	Average Precision
AUC	Area Under the Curve
CUSUM	Cumulative Sum (change-point statistic)
FPS	Frames per Second
GPU	Graphics Processing Unit
LLM	Large Language Model
LRU	Least Recently Used
MIL	Multiple-Instance Learning
PB	Petabyte
ROC	Receiver Operating Characteristic
TB	Terabyte
UCF	University of Central Florida
UCSD	University of California, San Diego
VAD	Video Anomaly Detection
VLM	Vision-Language Model

Stream and detection

x_t	video segment arriving at time t
L	segment length, in frames
s_t	anomaly score produced by the Sentinel
u_t	evidential uncertainty produced by the Sentinel
y_t	ground-truth label, unobserved by the system

Escalation and budget

D_t	cumulative-sum change-point statistic
g_t	escalation utility
a_t	escalation decision for segment t
$\hat{a}_t$	charged accelerator work
λ_t	shadow price of escalation
$\tilde{\lambda}_t$	price including the congestion surcharge
B	escalation budget, fixed by the hardware
q_t	backlog awaiting the expensive stage
N	cameras sharing one accelerator

Evidence and reasoning

$\mathcal{E}_t$	structured evidence list
$\mathcal{M}$	Scene Memory
$\mathcal{R}_t$	exemplars retrieved from Scene Memory
v_t, c_t, r_t	Reasoner verdict, confidence, explanation text

Output, cost and latency

$\hat{p}_t^{\text{prov}}$	provisional calibrated probability
$\hat{p}_t^+$	amended calibrated probability
Δ_{hard}	deadline for the probability
Δ_{soft}	deadline for the explanation
$\bar{C}$	long-run average cost per segment
$L_{\text{hard}}, L_{\text{soft}}$	latency of the probability and of the explanation
ρ, h	realized escalation rate, memoization hit rate

Symbols local to a single derivation are defined where they occur, and Table 3.1 in Chapter 3 lists the complete set used in the methodology.

*Dedicated to our parents, who inspired us and helped us
at every step.*

Chapter 1

Introduction

1.1 Motivations

The widespread deployment of surveillance cameras in public spaces, transportation systems, industrial environments, residential areas, and other safety-critical settings has resulted in a continuous growth of video data. Large surveillance infrastructures can generate substantial amounts of video that must be monitored over long periods of time. For example, Xu *et al.* reported in 2019 that a single camera recording at 720p and 30 frames per second could generate as much as 24 GB of encoded video per day [50]. This example illustrates the data volume associated with continuous recording, even for a single camera. Manual monitoring across many cameras is therefore costly, time-consuming, and susceptible to missed events and human error.

The difficulty is further amplified by the nature of surveillance video itself. Most recorded footage contains routine activities, while only a small portion corresponds to events that may require attention. Figure 1.1 provides a concrete example of this imbalance in UCF-Crime[44], where normal segments substantially outnumber abnormal segments, as reported by Acharya *et al.* [1].

This strong imbalance between normal and abnormal content makes continuous manual inspection inefficient and motivates automated mechanisms for identifying

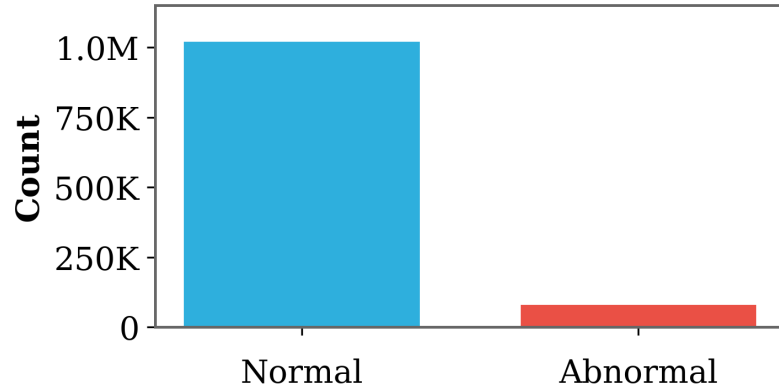


FIGURE 1.1: Imbalanced distribution of normal and abnormal segments in the UCF-Crime dataset [44]. Reproduced from Acharya *et al.* [1]; the distribution was computed by the original authors, not in this thesis.

unusual events. Video Anomaly Detection (VAD) addresses this problem by identifying activities or events that deviate from the expected patterns of a scene. VAD has consequently become an important research problem in computer vision, with applications ranging from video surveillance to autonomous driving and industrial inspection [4, 10, 41, 44].

Figure 1.2 illustrates the basic concept of VAD.

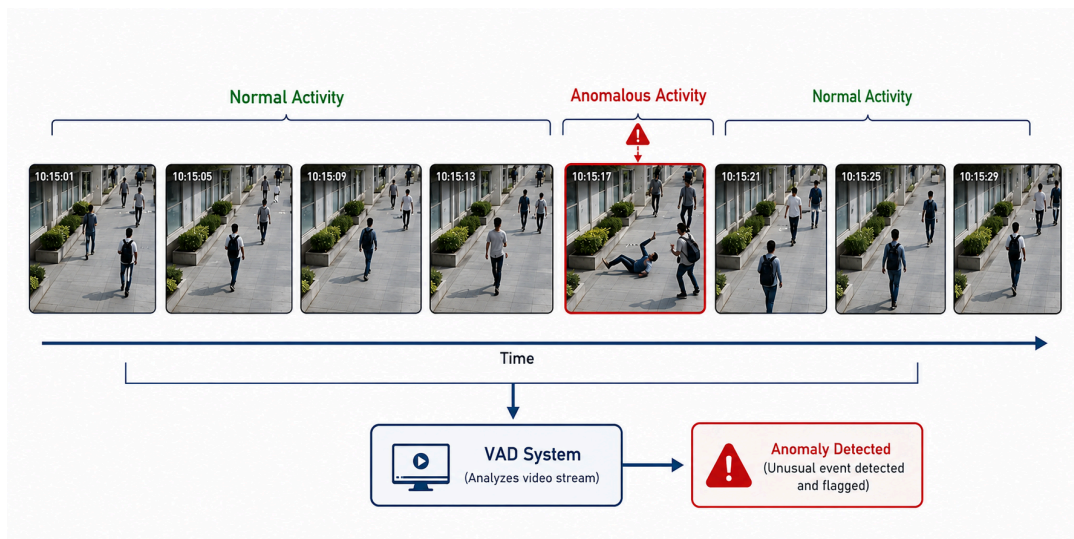


FIGURE 1.2: Conceptual illustration of video anomaly detection in a pedestrian scene. Normal walking activity is interrupted by a person falling, highlighted by the red-bordered frame, before normal activity resumes. The VAD system analyzes the video stream over time and flags the unusual event.

Early VAD methods primarily focused on learning the regular patterns of normal behavior and identifying deviations from them using reconstruction, prediction, or learned feature representations [20]. However, obtaining detailed temporal annotations for anomalous events is expensive because anomalies are often rare, diverse, and difficult to define precisely in long surveillance recordings. This limitation encouraged the development of weakly supervised and unsupervised approaches that reduce the dependence on detailed frame-level annotations. In particular, Sultani *et al.* [44] formulated VAD as a multiple-instance learning problem using only video-level labels and introduced the UCF-Crime dataset, which subsequently became an important benchmark for large-scale surveillance anomaly detection. This supervision-based distinction should not be conflated with whether a model requires task-specific parameter training. LAVAD, MoniTor, and PANDA use pretrained foundation models for training-free VAD [51, 54, 56]. They are therefore discussed separately from methods that learn an anomaly detector from unlabeled target videos. Likewise, AnomalyRuler is training-free in terms of model parameters but still uses normal reference examples [52].

A major shift occurred with the introduction of large-scale surveillance benchmarks and weakly supervised learning. In particular, Sultani *et al.* [44] introduced UCF-Crime and formulated VAD as a multiple-instance learning problem, where an untrimmed video is treated as a bag of temporal segments and only video-level labels are required during training. This formulation significantly reduced the annotation burden and established a widely adopted direction for real-world VAD. Subsequent methods improved the representation learning, temporal modeling, and ranking mechanisms of weakly supervised VAD, achieving increasingly strong anomaly detection performance.

Despite these advances, conventional VAD systems have largely focused on answering a single question: *“Is this video segment anomalous?”* Their output is typically an anomaly score or probability. Such a score is useful for ranking suspicious segments, but it provides little information about *what happened*, *which entities were involved*, or *why the event should be considered abnormal*. This limitation becomes particularly important in surveillance applications, where an operator must often make a decision

based on an alert. A high anomaly score without an interpretable reason can increase the cognitive burden on the operator and make it difficult to verify whether the system has correctly understood the event.

The emergence of large-scale vision-language models (VLMs), such as CLIP [39] and LLaVA [31], together with large language models (LLMs), such as GPT-3 [7] and Llama 3 [18], has created a new opportunity to address this limitation. Unlike conventional VAD models that primarily produce numerical anomaly scores, foundation models can connect visual observations with natural-language concepts. Recent studies have therefore explored the use of VLMs and LLMs for video anomaly detection and explanation.

For example, LAVAD [56] demonstrated that LLMs can be used for training-free VAD by converting video frames into textual descriptions and subsequently reasoning over the temporal sequence of captions. VAD-LLaMA [32] further explored the integration of large language models with VAD to provide natural-language explanations and temporal reasoning. Other approaches, such as AnomalyRuler [52], demonstrated that language models can induce human-readable rules from normal observations and use these rules to reason about anomalous activities. More recent systems such as Holmes-VAU [58] and PANDA [54] have further investigated large multimodal models for hierarchical understanding, reasoning, and generalization across diverse anomaly scenarios.

These developments represent an important conceptual transition in VAD: the objective is no longer limited to detecting an anomaly, but is gradually moving toward *understanding and explaining* anomalous events. However, this improvement introduces a fundamental systems problem. Large VLMs and LLMs are substantially more computationally expensive than conventional VAD models. Running such models continuously over every frame or every temporal segment of a surveillance stream is impractical for real-time deployment, particularly on resource-constrained hardware.

The need for computational efficiency has consequently motivated research into real-time and resource-aware VAD. REWARD [23] explicitly investigated real-time

weakly supervised VAD by reducing the temporal decision period and optimizing inference for streaming operation. However, its output remains an anomaly score without natural-language explanation. Similarly, VadCLIP [49] demonstrated that frozen vision-language representations can provide an efficient route to semantic anomaly detection, but it still relies on target-domain training and does not generate free-form explanations.

More recently, a promising direction has emerged that combines a *fast detector* with a *slow, expensive reasoning model*. Instead of applying a VLM to the entire video, these systems first use a lightweight model to continuously screen the stream and invoke the expensive model only for suspicious segments. SlowFastVAD [15], for example, combines a fast anomaly detector with selective VLM reasoning, while Holmes-VAU [58] uses anomaly-focused temporal sampling to reduce the number of frames processed by a multimodal model. Flashback [27] takes an even more resource-aware approach by moving expensive language-model processing offline and performing lightweight retrieval during online inference. MoniTor [51] also investigates training-free online VAD using large models over continuous streams.

These approaches demonstrate that selective reasoning is a promising way to reconcile the semantic capabilities of foundation models with the computational requirements of surveillance systems. Nevertheless, important limitations remain. First, the expensive reasoning model can still become part of the response path for an escalated event, making the time required to produce an alert dependent on the latency of the large model. Second, existing approaches generally control expensive computation using thresholds or heuristic selection strategies. A threshold determines *which* segments are escalated, but it does not directly specify *how much computation the deployment can afford*. Consequently, the fraction of the stream processed by the expensive model may change substantially when the scene, camera, workload, or hardware changes. Third, most evaluations report model latency or throughput after the system has been designed, rather than treating latency as an explicit operational constraint that the system must satisfy continuously.

These limitations become even more significant in practical surveillance environments. A real surveillance system may receive streams from multiple cameras while sharing a single accelerator or edge device. If several cameras simultaneously generate suspicious events, a selective VLM pipeline can experience a burst of expensive requests, resulting in a growing queue and increasing response latency. Therefore, merely reducing the average number of VLM calls is insufficient. A practical system must control the rate of expensive computation, handle bursts, and ensure that the initial anomaly alert is produced within a predictable time.

Furthermore, computational efficiency alone does not guarantee trustworthy explanations. A language model can produce a fluent and plausible explanation that is not actually supported by the visual evidence. Existing explanation-oriented VAD systems generally evaluate the quality of generated text, but a fluent explanation does not necessarily imply that the explanation is grounded in the observed video. This creates a second deployment problem: an operator needs not only an explanation, but also a way to determine whether the explanation is supported by independently observable evidence.

Thus, practical VAD requires addressing several requirements simultaneously. The system should continuously monitor a video stream in real time, operate within a limited computational budget, provide an anomaly decision without waiting for a large model, and produce an explanation that is grounded in the actual visual evidence. Existing methods address these requirements individually or in partial combinations, but the complete intersection remains insufficiently explored.

Motivated by these observations, this thesis aims to develop **ARGUS: An Adaptive, Resource-Aware, Grounded Understanding System for Real-Time Explainable Video Anomaly Detection**. Rather than treating the fast/slow cascade as a simple heuristic switch, ARGUS formulates it as a *budgeted and deadline-aware scheduling problem*. A lightweight causal detector continuously monitors the incoming stream, while an expensive vision-language reasoner is invoked selectively when additional semantic evidence is expected to be valuable.

The proposed framework addresses the problem from four complementary perspectives. First, it introduces a *two-deadline response mechanism*, in which a calibrated anomaly score is produced by the lightweight detector under a hard latency deadline, while the natural-language explanation is generated asynchronously under a softer deadline. Consequently, the expensive VLM is not placed on the critical path of the initial alert. Second, ARGUS uses a *budgeted escalation controller* that dynamically regulates access to the expensive reasoning path according to an explicit computational budget rather than relying solely on a fixed anomaly threshold. Third, the framework incorporates several mechanisms for reducing the latency and redundant computation of the expensive path, including speculative escalation, prompt-prefix caching, and evidence-based memoization. Fourth, an *evidence grounding mechanism* independently extracts objects, actions, and motion attributes from the escalated video segment and requires the VLM’s explanation to explicitly refer to this evidence.

1.2 Objectives

The overall objective is to develop and evaluate ARGUS, a visual-only framework for video anomaly detection on resource-constrained surveillance systems. It aims to provide timely anomaly probabilities and evidence-grounded explanations to support human operators. The specific objectives are:

1. To train a lightweight detector on video-level labels alone, one that scores each arriving segment from current and past frames only.
2. To control access to the expensive vision-language model through an explicit compute budget, including when several cameras share an accelerator, and to cut both the cost and the latency of that model’s use through frame caching, prefix caching, memoization and speculative escalation.
3. To produce a calibrated anomaly probability within the initial alert deadline and generate explanations asynchronously within a separate deadline, without making the initial alert wait for the reasoning stage.

4. To generate natural-language explanations using a frozen vision-language model supplied with independently extracted visual evidence, and to check that the explanations refer to valid evidence entries.
5. To adapt the reasoning stage to individual camera scenes through a bounded memory of normal reference clips and subsequent observations, without updating the pretrained models' weights.
6. To evaluate detection accuracy, probability calibration, deadline attainment, compute cost, and explanation faithfulness on UCF-Crime and the visual component of XD-Violence. Ablation studies and comparisons also test scene adaptation and budget allocation across several camera streams.

1.3 Contributions

Recent video anomaly detection systems pair a fast detector with a slow, expensive one [15, 27, 51, 53, 58]. This work treats that cascade as a scheduler rather than a switch, one that works under a stated compute budget and a stated latency contract. Each mechanism below is derived from one of the two constraints that formulation imposes, rather than introduced as an independent component.

1. **A formulation of selective video anomaly detection as a budgeted, deadline-aware scheduling problem.** Chapter 3 states the design as the minimization of a proper scoring rule under two bounds. One limits the long-run fraction of segments that may reach the expensive model. The other limits how often the numeric decision misses its deadline.
2. **A budgeted escalation controller that holds a compute budget without per-deployment tuning.** Access to the expensive path is priced by a multiplier adapted online by dual ascent, so that the compute charged against the accelerator tracks a budget fixed by the hardware. Nothing is calibrated on a validation set the deployment may not resemble. The same price extends without modification

to several cameras sharing one accelerator, a regime the surveyed literature [15, 27, 51, 53, 58] does not address.

3. **A two-deadline output contract that removes the large model from the critical path.** A calibrated anomaly probability is emitted for every segment within a hard deadline using the streaming detector alone, and the explanation, when it arrives, amends that alert in place. Time-to-alert therefore does not depend on the vision-language model, on the network, or on queue depth, and a failure of the expensive path costs an explanation rather than an alert.
4. **Mechanisms that reduce the duration of the expensive path rather than only its frequency.** Speculative escalation begins the slow path before the trigger fires, a per-camera prompt-prefix cache removes constant context from the per-call cost, and evidence-hash memoization suppresses repeated calls within a single event. Prior cascaded systems reduce how often the expensive model runs. The three above reduce how long it takes when it does.
5. **A grounding contract that makes explanation faithfulness automatically checkable.** Visual evidence is extracted independently of the language model, and the explanation is required to cite entries of that evidence list. Citations are checked automatically, a single re-prompt is issued on failure, and ungrounded output is suppressed, so that an unsupported explanation becomes a detectable event rather than a matter of opinion.
6. **An explicit cost and latency model, and an evaluation protocol built to falsify it.** Chapter 3 states the expected per-segment cost and the two deadline budgets in closed form. Chapter 5 reports every constituent term separately, alongside accuracy, calibration and explanation faithfulness, so that no axis can be improved by silently degrading another.

Several components are adopted rather than invented, and each is credited where it appears. They are the cascade structure itself [15], the multiple-instance training signal [44], the evidential uncertainty head [42], the change-point statistic [37], the detector and tracker the Grounder runs [11, 59], and prefix reuse in language-model

serving [26]. This work claims the formulation above, the mechanisms it makes possible, and the evaluation that tests whether they hold.

1.4 Challenges

Four difficulties shape the design that follows, and each is met by a specific mechanism rather than by a general appeal to efficiency.

Cost that cannot be predicted before deployment. A threshold tuned to escalate a given fraction of a validation set escalates some other fraction of a live camera whose scores are distributed differently. Nothing reveals the true figure until the accelerator saturates. What a deployment can afford is a rate, whereas a threshold fixes a boundary, and the two coincide only on the data the threshold was tuned on.

Latency that is worst when it matters most. In a cascade whose alert waits for the large model, the system responds fastest when nothing is happening and slowest while an event is in progress. Its worst case is the slow path’s worst case, which on shared or networked hardware is effectively unbounded.

Explanations that cannot be checked. A fluent sentence about a video carries no evidence that it describes that video. Judging whether a generated explanation is faithful ordinarily requires a human reader, which cannot be done at the rate alerts are produced, and a second model brought in to judge would itself require verification.

Scenes the training data never contained. Constant forklift movement is ordinary at a loading dock and anomalous in a corridor at night. No public benchmark contains the camera a system is eventually deployed on, so a notion of normality learned from a corpus is necessarily approximate at the installation that matters.

A fifth constraint is practical rather than technical. Every component must be trainable or runnable within the resources available to this project. That rules

out an instruction-tuning corpus of the scale used by recent fine-tuned surveillance assistants, and it is why the reasoning model is left frozen.

1.5 Organization

The remainder of this thesis is organized as follows. Chapter 2 covers related work and the limitations of existing systems. Chapter 3 describes the proposed framework and its main components. Chapter 4 outlines the implementation choices, configuration, and development plan. Chapter 5 presents the evaluation protocol, including datasets, metrics, baselines, and ablation studies. Chapter 6 concludes the thesis and discusses deployment considerations, ethical issues, and future work.

Chapter 2

Related Works

This chapter reviews Video Anomaly Detection (VAD) methods relevant to continuous surveillance with limited computing resources and human-readable explanations. The comparison considers six properties: real-time or online operation, explanation, selective computation, use of vision-language or language models, freedom from target-domain training, and demonstrated operation on resource-constrained hardware.

The papers contribute different parts of the proposed design rather than interchangeable solutions. SlowFastVAD, AssistPDA, MoniTor, and Flashback are the closest system-level references, but they use different strategies for handling expensive inference [15, 27, 51, 53]. The remaining papers inform the lightweight detector, temporal context, scene adaptation, or explanation stage. The discussion distinguishes demonstrated capabilities from requirements that remain to be tested in this thesis.

2.1 Video Anomaly Detection

VAD identifies events that depart from expected activity. Its difficulty comes from both the rarity of anomalies and the dependence of normal behavior on the scene. Learning normal temporal patterns provides one approach, while learning from videos labeled normal or anomalous provides another [20, 44]. For the present thesis,

the main task is to score an incoming video stream and explain selected events without examining future segments.

The reviewed papers use different supervision settings. Sultani et al. and REWARD learn from video-level labels, which do not identify the exact anomalous frames [23, 44]. AnomalyRuler instead uses a small normal-only reference set [52]. Holmes-VAU uses frame-level supervision for its anomaly scorer and instruction data for its multimodal model [58]. These differences matter when comparing annotation requirements and detection performance.

Three deployment properties also need to be separated. Online processing means that a method can consume a stream without requiring the complete future video. Throughput measures how much video can be processed per unit time. Response delay measures how long an operator waits for a result. MoniTor and REWARD illustrate why a frequent score update or a high processing rate does not, by itself, establish a short end-to-end alert delay [23, 51].

Similarly, producing an anomaly category, retrieving a descriptive caption, and generating an explanation are different outputs. VadCLIP supplies category-related information, Flashback retrieves captions, and systems such as VAD-LLaMA generate textual responses [27, 32, 49]. Their explanatory capabilities should therefore be assessed according to what the operator actually receives.

2.2 Traditional VAD

Traditional approaches provide the background to the learned detectors reviewed here: visual or motion features are compared with a model of normal activity, and deviations produce an anomaly score. Their relevance to this project is the idea of maintaining a compact representation of expected behavior rather than enumerating every possible anomaly. Hasan et al. develop the temporal-regularity principle using learned representations [20], while Sultani et al. compare their weakly supervised detector with earlier sparse-coding and autoencoder baselines [44].

The reading set concentrates on recent deep-learning and foundation-model methods, so traditional techniques serve as background rather than the principal comparison group. A compact detector remains useful as an always-on front end, but a numerical deviation score alone does not supply the natural-language account required by this thesis.

Nor should a simple model be assumed to be real-time without measuring feature extraction and temporal buffering. The distinction between an efficient detector and an efficient complete pipeline remains relevant to the recent methods as well.

2.3 Deep Learning-based VAD

The deep-learning papers in the reading set primarily inform the Sentinel, the proposed lightweight detection stage. They address representation learning, video-level supervision, and the identification of anomalous snippets, but do not all address streaming explanation.

Sultani et al. [44] introduced UCF-Crime and a deep multiple-instance learning formulation. Each video is divided into 32 segments; a ranking loss encourages higher scores in anomalous videos, with sparsity and temporal-smoothness terms. This is a foundational reference for learning from video-level labels. However, its whole-video segmentation is not a streaming protocol, and its output is an anomaly score rather than an explanation.

VadCLIP [49] retains a frozen CLIP backbone and learns lightweight temporal adaptation and prompts. Its classification and alignment branches support binary anomaly detection and category-related localization. It offers a relatively low-parameter route to semantic features, but still requires weakly labeled target-domain data. Whole-video temporal modeling and the absence of a real-time latency evaluation limit what can be concluded about live deployment; category labels are also not free-form explanations.

The attention-driven pseudo-label method of Yang et al. [55] synchronizes pseudo-label generation with self-training. Attention-based snippet processing, denoising, and multi-scale temporal interaction aim to reduce error accumulation in weakly supervised learning. Its relevance is improved training of the lightweight detector, not selective language-model inference: the reviewed evaluation does not establish generated explanations, streaming latency, or edge operation.

Veesam et al. [47] combine temporal graph attention, transformer-augmented recurrence, multimodal reconstruction, and few-shot components. This offers ideas for temporal and cross-camera context, but is less directly aligned with the present visual-only design because it also considers audio and motion-sensor information. The reading summary does not establish a comparable UCF-Crime or XD-Violence real-time evaluation, nor a natural-language explanation stage, for this system.

These methods motivate candidate detector components rather than additions that must all be implemented. Any adopted component must be tested for causal operation, detection quality, and the cost it adds to the always-on stage.

2.4 Zero-shot and Training-free VAD

Training-free VAD uses pretrained models without target-domain parameter updates. It is distinct from both the supervision used during pretraining and the reference information supplied at inference. A method can leave its model weights fixed while still requiring examples of normal activity, as AnomalyRuler does [52].

LAVAD [56] establishes the caption-based route: pretrained models describe frames, clean noisy captions, summarize temporal windows, and assign anomaly scores, followed by cross-modal score refinement. It demonstrates semantic anomaly detection without training a new detector. However, the reviewed pipeline is offline, involves repeated captioning and language-model calls, and remains sensitive to caption and prompt quality. It is a reference for semantic reasoning rather than an end-to-end real-time baseline.

AnomalyRuler [52] first induces normality and anomaly rules from a few normal examples, then applies those rules to test frames to obtain verdicts and rationales. Rule aggregation, smoothing, and rechecking improve robustness. This supports the thesis’s interest in adaptation without fine-tuning, but the normal reference set still matters. Its one-class evaluations and per-frame reasoning do not establish real-time performance on long, weakly supervised crime benchmarks.

PANDA [54] extends training-free reasoning through scene-aware retrieval, heuristic prompting, tool-assisted reflection, and a chain of memory. It is relevant to scene-specific knowledge and adaptation across different situations. Its emphasis is generalist detection, however, and the reviewed work does not provide a real-time streaming or edge evaluation. Reflection and tool use must not be treated as cost-free merely because parameter training is avoided.

MoniTor and Flashback also avoid target-domain parameter training, but solve the inference problem differently: MoniTor maintains online textual context, whereas Flashback performs language-model generation offline and retrieves from the resulting memory online [27, 51]. Their execution strategies are examined below.

Together, these papers show that training requirements, normal-reference requirements, and inference cost are separate comparison axes. None can be inferred from the term “training-free” alone.

2.5 Vision-Language Models for VAD

Vision-Language Models (VLMs) and Large Language Models (LLMs) extend VAD beyond a numerical score. The reviewed systems use captions, learned video–language alignment, instruction tuning, or retrieval to relate observations to a textual account. These mechanisms differ in training needs and in how closely their output is tied to the observed event.

VAD-LLaMA [32] combines a video anomaly detector with long-term context and a language-model adaptor. Lists of normal, anomalous, and recent clip features

provide context, while staged training links detection to textual explanations and dialogue. It is therefore relevant to both memory and explanation. Nevertheless, it requires domain-specific training and adaptor instruction tuning; updating a context list online is not evidence that the full system meets a real-time response deadline. The reading summary also identifies limited systematic evaluation of explanation quality.

Holmes-VAU [58] combines an anomaly scorer with an anomaly-focused temporal sampler and a tuned multimodal model. Its HIVAU-70k instruction data supports clip-, event-, and video-level judgments, descriptions, and analysis. Sampling anomaly-rich frames reduces multimodal input and first-token latency in the reported configuration. However, selecting frames from a complete video is different from deciding causally on an incoming stream. Its frame-level scorer supervision and instruction tuning also distinguish it from training-free methods.

AssistPDA [53] targets online prediction, detection, and interactive analysis. It distills temporal relations into a streaming module and uses a tuned language model to decide when to respond. Unlike an offline sampler, it explicitly addresses continuing video input, but requires purpose-built instruction data and substantial hardware. Its execution characteristics are especially relevant to the present thesis and are discussed in the next section.

LAVAD and AnomalyRuler demonstrate explanations through captions or rules, while Flashback returns a nearby pre-generated caption [27, 52, 56]. These are useful forms of interpretation, but they provide different evidence of faithfulness. Text-quality metrics, a convincing rationale, or a retrieved description do not establish that each claim is supported by independently detected entities and actions.

The relevant design question is therefore not simply whether a VLM is present. It is how the model receives temporal context, when it runs, what supervision it needs, and how its output can be checked without delaying the initial alert.

2.6 Existing Relevant Methods

The reading summary ranks SlowFastVAD, AssistPDA, MoniTor, and Flashback as the closest references. They approach the thesis’s combined objectives from different directions: selective expensive inference, a streaming assistant, online caption-based scoring, and offline generation with online retrieval. They should not all be described as the same fast/slow cascade.

SlowFastVAD. This method [15] is the closest architectural reference. A lightweight detector supplies scores, and an entropy-based intervention selects ambiguous subsequences for a retrieval-augmented VLM. A knowledge base supplies normal patterns and inferred abnormal patterns, and the fast and slow scores are fused. The reported speed of the lightweight branch must be distinguished from end-to-end throughput: the summary identifies no complete streaming-latency evaluation. Cloud-model dependence, per-scene knowledge construction, and limited-scene one-class evaluations also leave resource-constrained deployment and real-world weakly supervised evaluation open.

AssistPDA. AssistPDA [53] uses spatio-temporal relation distillation, cached temporal context, and a streaming response token to support anomaly prediction, detection, and analysis. The summary reports approximately 15–20 frames per second on an A6000. This is useful evidence of online operation, not evidence of low-resource deployment. The model requires LoRA instruction tuning and the VAPDA-127K data pipeline. Its prediction and text-quality evaluations also differ from frame-level AUC/AP comparisons with conventional detectors. These differences motivate evaluating detection, explanation, and timing together rather than selecting whichever metric favors a particular system.

MoniTor. MoniTor [51] combines frame captioning, caption filtering, language-model summarization, anomaly priors, dynamic memory, and a scoring queue. It avoids task-specific training and maintains context over an incoming stream. The summary distinguishes a score interval of approximately 0.6 seconds from an end-to-end delay of approximately 6.5 seconds. Multiple model operations are still

performed per frame, so it is not a lightweight screen that invokes a VLM only on rare events. Scene transitions and missing fine-grained visual details remain important failure cases; the summary does not establish edge operation or a separate explanation-quality evaluation.

Flashback. Flashback [27] moves language-model generation into an offline Recall stage, where normal and anomalous captions form a pseudo-scene memory. During online Respond, video embeddings retrieve matching captions to obtain scores and a human-readable description. The summary reports 42 frames per second on a commercial GPU and processing below one second per one-second segment. This is an important resource-aware alternative to online VLM generation. Its explanation is nevertheless selected from a pre-generated collection, not newly generated from independently extracted evidence. Memory coverage, independent segment scoring, and prompt sensitivity limit the temporal and explanatory conclusions that can be drawn.

REWARD. REWARD [23] is the strongest real-time weakly supervised reference in the reading set. It derives segment pseudo-labels from video-level supervision and fine-tunes a detector on sampled raw frames. Its reported 63 frames per second on an RTX-2070 accompanies a 32-frame window sampled at 5 frames per second and an approximately 6.9-second response delay. The distinction is important for setting a meaningful alert deadline. REWARD supplies anomaly scores rather than language explanations, requires target-domain training, and uses fixed sampling that can lose short events.

VAD-LLaMA, Holmes-VAU, and AnomalyRuler provide complementary ideas for memory, selective temporal sampling, and normal-reference reasoning [32, 52, 58]. LAVAD and PANDA supply broader semantic and adaptive reasoning approaches, while VadCLIP, Sultani et al., and the two lower-ranked detector papers inform the lightweight stage [44, 47, 49, 54–56]. This ranking reflects relevance to the thesis, not a single accuracy leaderboard.

All numerical timings above describe the configurations summarized in the reading document. Different hardware, input rates, temporal windows, and offline stages

prevent treating those figures as directly comparable measurements of one common deployment.

2.7 Comparison of Existing Approaches

Table [2.1](#) reproduces the capability matrix from the reading summary, preserving its paper order, venue descriptions, and six capability ratings. The ranking measures proximity to the combined goal of real-time, explainable, resource-aware surveillance, not publication quality or detection accuracy alone. A partial capability must be read together with the method-specific limitations discussed above.

TABLE 2.1: Capability matrix (papers ordered by relevance)

#	Paper (venue, year)	RT/ Online	Explain	Selective	VLM/ LLM	Train- free	Edge
1	SlowFastVAD (arXiv, 2025)	~	✓	✓	✓	~	×
2	AssistPDA (arXiv, 2025)	✓	✓	✓	✓	×	×
3	MoniTor (NeurIPS, 2025)	✓	~	~	✓	✓	×
4	Flashback (arXiv, 2025)	✓	✓	✓	✓	✓	✓
5	REWARD (Karim et al.) (WACV, 2024)	✓	×	✓	×	×	✓
6	VAD-LLaMA (Lv & Sun) (2024)	~	✓	~	✓	×	×
7	Holmes-VAU (Zhang et al.) (CVPR, 2025)	×	✓	✓	✓	×	×
8	AnomalyRuler / “Follow the Rules” (ECCV, 2024)	×	✓	~	✓	~	×
9	LAVAD (Zanella et al.) (CVPR, 2024)	×	✓	×	✓	✓	×
10	PANDA (Yang et al.) (NeurIPS, 2025)	×	~	~	✓	✓	×
11	VadCLIP (Wu et al.) (AAAI, 2024)	×	×	✓	✓	×	~
12	Sultani et al. (CVPR, 2018)	×	×	✓	×	×	n/a
13	TGAT + Transformer-RNN (Veesam et al.) (Sci. Rep., 2025)	×	×	×	×	×	×
14	Attention-driven pseudo-label (Yang et al.) (Pattern Recognit., 2026)	×	×	~	×	×	×

Legend: ✓ the paper has this capability; ~ it has this capability partially or in a limited form; × it does not have this capability; n/a the axis does not meaningfully apply to that paper. Rows are ordered by relevance, that is, by how closely a paper approaches the thesis’s combined goal of being real-time, explainable, resource-aware and edge-deployable at the same time, so paper 1 is the closest match and paper 14 the least.

The highest-ranked group covers four different execution strategies rather than four identical cascades. The next group contributes specific strengths: REWARD for real-time weak supervision, VAD-LLaMA for explanation and context memory, Holmes-VAU for anomaly-focused sampling, and AnomalyRuler for normal-reference rules. LAVAD, PANDA, VadCLIP, and Sultani et al. supply the semantic and detection foundations. The final two papers have narrower relevance to temporal modeling and pseudo-label learning.

The matrix uses “selective” broadly, including temporal sampling, efficient adaptation, and moving expensive computation offline. It does not mean that every marked method gates an online VLM behind a cheap detector. Likewise, a positive explanation rating can include a retrieved caption, and the Edge column follows the summary’s broad inclusion of commercial hardware. These qualifications prevent a capability mark from being mistaken for evidence of the complete deployment required here.

The central limitation identified in the reading summary is the absence, within these fourteen papers, of a demonstrated combination of real-time resource-constrained operation, generated and grounded explanations, selective computation, and joint detection, explanation-quality, and latency evaluation on realistic surveillance benchmarks. This is a gap in the reviewed evidence, not a claim that every method lacks each capability individually.

2.8 Research Gap

The reviewed work already provides strong precedents for each major component of the proposed system. The research gap is their integration under a measurable deployment constraint, rather than the introduction of the fast/slow idea itself. The following limitations follow from the specific methods summarized above.

First, streaming capability and prompt response are not equivalent. REWARD and MoniTor illustrate the contribution of temporal windows and pipeline delay, while AssistPDA demonstrates online interaction on hardware different from a low-resource deployment [23, 51, 53]. A useful comparison must time the initial anomaly decision separately from the explanation and measure the complete pipeline rather than only its fastest component.

Second, reducing expensive work does not itself define an enforceable hardware budget. SlowFastVAD selects uncertain windows, Holmes-VAU reduces sampled frames, and Flashback removes online language-model generation [15, 27, 58]. These

are useful but different optimizations. The reading summary does not establish an explicit shared-workload budget with controlled response deadlines across competing live streams. This motivates examining budget allocation and queue behavior directly.

Third, adaptation requirements remain important. VAD-LLaMA, Holmes-VAU, and AssistPDA require training or instruction tuning, whereas AnomalyRuler needs representative normal examples and Flashback depends on the coverage of generated memory [27, 32, 52, 53, 58]. A scene-aware reasoning stage therefore needs an explicit account of how its reference information is obtained and how errors in that information are handled. The present thesis does not claim that its entire detector is training-free.

Fourth, explanation evaluation is incomplete or different across the papers. The summary identifies absent systematic explanation evaluation for SlowFastVAD and VAD-LLaMA, text-quality evaluation for AssistPDA, and retrieval-limited descriptions for Flashback [15, 27, 32, 53]. These observations motivate both checks against independently extracted evidence and a separate assessment of whether the resulting explanation is faithful. Valid evidence identifiers alone cannot verify that an object detection or an inference is correct.

Finally, the benchmark and hardware differences prevent a single ranking of published scores from answering the deployment question. Results on one-class scene datasets, real-world weakly supervised datasets, and instruction-answering tasks measure different capabilities. The thesis therefore needs matched operating points and separate measurements of detection, calibration, compute, response time, and explanation quality.

The resulting research requirements are:

- causal monitoring with the initial score separated from slower explanation;
- explicit control of expensive work under a shared computing budget;
- scene-specific reasoning without instruction tuning the large model;

- generated explanations whose evidence references can be checked; and
- joint evaluation of accuracy, resource use, deadline attainment, and explanation faithfulness.

These requirements define what the proposed framework must demonstrate. They do not establish its superiority in advance; comparisons with the closest reviewed systems and ablations of each proposed mechanism are needed to test the claimed benefits.

2.9 Summary

This chapter reviewed video anomaly detection methods, focusing on their supervision requirements, computational efficiency, real-time operation, and explanation capabilities. The comparison motivates a framework that considers detection accuracy, computing cost, response time, and explanation faithfulness together. Chapter [3](#) presents the proposed framework and explains how its components address the identified research gap.

Chapter 3

Proposed Methodologies

3.1 Problem Formulation

3.1.1 Operating setting

We consider a fixed camera producing a continuous encoded video stream. The input is visual only. Many deployed installations record no audio, so a system that depended on an audio track could not be used on a large share of the cameras it is intended for. One of the benchmarks used in Chapter 5, XD-Violence [48], does carry an audio track alongside its video. Its video is used and its audio is not, so that results on it are comparable with visual-only methods rather than with audio-visual ones.

The formulation is stated for one camera throughout, since every mechanism operates on a single stream. Deployments in which several cameras share the hardware, and those holding more than one accelerator, require no change to any of them and are treated in Section 3.4.6.

The stream is partitioned into short segments $x_1, x_2, \dots$ of L frames each, taken every $L/2$ frames, so that consecutive segments overlap by half their length, as is usual for this task [44]. At 25 fps with $L = 16$, a segment arrives every 0.32 s.

Processing is constrained to be causal: when segment x_t arrives, only $x_1, \dots, x_t$ may be consulted. A live feed provides no look-ahead, so any operation requiring the complete sequence, such as normalization of scores over a whole video or a bidirectional temporal model [15], lies outside the setting considered here.

Two resources are limited in this setting, and both are treated as design constraints rather than as outcomes measured after the system is built. The first is *compute*. The accelerator at a deployment can sustain a large model on only a small fraction of the arriving segments. A segment routed to that model is said to be *escalated*. We write $B \in (0, 1)$ for the *escalation budget*, the fraction of arriving segments that may be escalated, aggregated over every stream the accelerator pool serves. A single value of B therefore serves a deployment of any size. The controller of Section 3.4 distributes that one rate among however many streams are present. The fleet update of Section 3.4.6 enforces it as a single constraint on the average across cameras rather than one per stream. Because B follows from the throughput of the hardware and the rate at which segments arrive, both known before deployment, it is a property of the installation rather than a hyperparameter of any dataset.

The second is *time*. What matters to an operator watching an alert queue is whether each one arrives in time, not what the delays average to. We therefore require the system to commit to two deadlines. A calibrated anomaly probability must be emitted for every segment within a hard deadline Δ_{hard} . For segments selected for detailed analysis, an explanation should follow within a soft deadline Δ_{soft} .

3.1.2 Supervision available

Training uses video-level labels only. This follows the weakly supervised protocol of realistic surveillance benchmarks, among them UCF-Crime [44] and XD-Violence [48]. A label says only that an anomaly occurs somewhere in the video. It does not mark the point in time. Frame-level annotations exist only in the test splits and are used exclusively for evaluation. We additionally assume that a short period of ordinary activity, on the order of a few minutes, can be observed at a target camera before

the system is relied upon. This requirement is readily satisfied in deployment and is what permits the scene adaptation of Section 3.6 to proceed without any labeling.

An instruction-tuning corpus pairs video clips with the responses a model should produce for them. None is assumed or constructed here. The reasoning stage must therefore operate zero-shot, which is what makes the grounding and retrieval mechanisms of Sections 3.5–3.7 necessary rather than optional. The choice itself is justified in Section 3.7.

3.1.3 Objective

A policy ϕ determines, for each arriving segment, whether that segment is escalated and what probability is reported for it. Let $a_t^\phi \in \{0, 1\}$ indicate escalation of segment t under ϕ , let $\hat{p}_t^\phi \in [0, 1]$ be the probability finally reported for it, and let $y_t \in \{0, 1\}$ be the true label, which is not observed at run time. Here $\hat{p}_t^\phi$ denotes the amended probability where an amendment arrives within Δ_{soft} , and the provisional probability otherwise. The distinction is developed in Section 3.8. With L_t^ϕ the latency of the numeric decision, the design problem is

$$\min_{\phi \in \Phi} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \ell(\hat{p}_t^\phi, y_t) \quad \text{subject to} \quad \begin{cases} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T a_t^\phi \leq B, \\ \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{I}[L_t^\phi > \Delta_{\text{hard}}] \leq \epsilon, \end{cases} \quad (3.1)$$

where Φ is the set of causal policies admissible under Section 3.1. For ℓ we use the Brier score [6] $\ell(p, y) = (p - y)^2$, a proper scoring rule [17], so that calibration is optimized alongside discrimination.

Both the objective and the constraints are expressed as long-run averages over the stream, which is the appropriate form for a system that runs continuously and for which no fixed test distribution exists. The first constraint bounds the compute the system consumes, and B is fixed by the installation as described above. It is written on escalations here and refined in Section 3.4 to the compute the accelerator actually spends, once speculative execution has been introduced. The second constraint

bounds the frequency with which the numeric decision misses its deadline. Here ϵ is set to 0.01, so that at most one segment in a hundred may exceed Δ_{hard} .

Equation 3.1 is a design criterion rather than a quantity optimized at run time. The label y_t is never available to the deployed system. It exists only on the annotated evaluation splits, where the objective is estimated, and Section 3.3.3 describes the weak supervision used to approximate it during training. What the equation fixes is the standard against which the design is judged, and the remainder of the chapter is organized around meeting its two constraints. The compute constraint is enforced by the controller of Section 3.4, which adapts a price by dual ascent so that the compute charged against the accelerator tracks B on any stream. The latency constraint is met by construction in Section 3.8, where the numeric decision is emitted from the streaming tier alone and therefore never waits on the reasoning stage. The components introduced between those two sections exist to make the accuracy attainable within those limits.

3.1.4 Notation

Table 3.1 lists every symbol this chapter uses.

TABLE 3.1: Symbols of the proposed method.

Symbol	Meaning
x_t	Video segment arriving at time t
L	Segment length in frames
T_s, T_ℓ	Short and long Sentinel buffer lengths, in segments
$s_t \in (0, 1)$	Sentinel anomaly score
$\bar{s}_t$	Running baseline of the score during quiet periods
$u_t \in (0, 1]$	Sentinel evidential uncertainty
$D_t, \tilde{D}_t$	Cumulative-sum change-point statistic, and its normalized form
σ_D	Scale at which D_t is squashed into $\tilde{D}_t$
$\sigma(\cdot)$	Logistic function, distinct from the scale σ_D above

Table 3.1 (continued)

Symbol	Meaning
δ	Slack term of the change-point statistic
γ	Update rate of the running baseline $\bar{s}_t$
μ_t	Confidence weight on the pseudo-label term of the training loss
$\hat{g}_t$	Estimated drift of D_t over recent segments
$g_t \in (0, 1)$	Escalation utility
$\omega \in (0, 1)$	Weight balancing the two terms of g_t
$\lambda_t \geq 0$	Shadow price of escalation
$\tilde{\lambda}_t$	Price with the congestion surcharge, used for decisions
B	Escalation budget, set by hardware
$a_t \in \{0, 1\}$	Escalation decision
$\hat{a}_t$	Charged compute: escalation plus wasted speculation
z_t	Indicator that the speculation begun at $t - H$ went unconfirmed
ν	Cost share at which a speculation is charged
η	Step size of the dual update on λ_t
κ, q^*	Gain and target of the queue coupling
$\phi \in \Phi$	Policy, and the set of admissible causal policies
ϵ	Tolerated rate of hard-deadline misses
q_t	Backlog of windows awaiting the Reasoner
w_t	Escalation window length
H, m	Speculation horizon and drift lookback, in segments
β	Sharpness of the speculation estimate $\hat{\pi}_t$
$\mathcal{W}, \Gamma_j$	Window-length set and its drift thresholds
$\hat{\pi}_t$	Estimated probability that escalation follows within H
$N, B_{\min}$	Cameras sharing an accelerator, and each camera's reserved rate
$\mathcal{E}_t$	Structured evidence list from the Grounder
$\tau, v_{\min}$	Grounder sampling interval, and the speed below which an entity is at rest

Table 3.1 (continued)

Symbol	Meaning
$\mathcal{R}_t$	Entries retrieved from Scene Memory for window t
k, θ_s	Number of entries retrieved, and the similarity threshold
h	Evidence-hash memoization hit rate
ρ, ρ_{spec}	Realized escalation rate, and speculation rate
$\bar{\pi}$	Mean success rate of speculation
Λ_{spec}	Lead time gained by speculation, at most H segments
$c_{\text{dec}}, c_{\text{sen}}, c_{\text{gr}}, c_{\text{rea}}$	Per-segment cost of decoding, the Sentinel, the Grounder and the Reasoner
$c_{\text{bb}}, c_{\text{head}}$	Backbone and temporal-head shares of c_{sen}
$\bar{C}$	Long-run average cost per segment
$L_{\text{hard}}, L_{\text{soft}}$	Latency of the score and of the explanation
$t_{(\cdot)}$	Measured duration of one stage, subscripted by stage name
$\mathcal{M}$	Scene Memory of (embedding, verdict) entries
$y_t \in \{0, 1\}$	Ground-truth label, unobserved by the system
v_t, c_t, r_t	Reasoner verdict, stated confidence, explanation text
$\tilde{c}_t$	Reasoner confidence after recalibration on held-out data
$\Delta_{\text{hard}}, \Delta_{\text{soft}}$	Deadlines for the score and for the explanation
$\hat{p}_t^{\text{prov}}, \hat{p}_t^+$	Provisional and amended calibrated probability
θ^n, θ^e, ψ	Fitted coefficients of the two fusion maps

3.2 System Overview

ARGUS is organized around two stages whose costs differ by orders of magnitude. The cheap stage is a small detector that examines every segment. The expensive stage is invoked on a small fraction of segments and comprises an evidence extractor, a retrieval step, and a vision-language model that reasons over both. Two further

components complete the system. The Escalation Controller governs access to the expensive stage, and the Fusion layer enforces the latency contract. The division between the two stages is adopted from recent fast/slow detectors, not introduced here [15, 27, 51, 53, 58]. Decoding and fusion are excluded from the two stages because neither carries appreciable computational cost.

What this work changes is the status of the cascade. In existing fast/slow systems it acts as a switch. A threshold decides which segments go to the expensive branch, but not how many. The number that go depends on the scene, so the cost of that branch is an outcome rather than a choice. In ARGUS it acts as a scheduler that allocates a fixed resource among competing demands. The threshold is replaced by a price, which the controller raises and lowers to hold the compute constraint of Eq. 3.1. The second constraint of Eq. 3.1, on latency, is held elsewhere, by the Fusion layer described at the end of this section.

Video enters as an encoded bitstream and is decoded into frames. Each frame is embedded once on arrival and retained in a small cache, so that the frames shared by overlapping segments are not embedded twice. The *Sentinel* (Section 3.3) then scores every segment from those cached embeddings. It works at two temporal scales, since brief and slow-developing events need different amounts of context. Alongside each score it reports an evidential uncertainty. The Escalation Controller (Section 3.4) converts the pair (s_t, u_t) into a scalar utility and compares it against a price λ_t , adapted online so that the compute charged against the accelerator tracks B . The same price also admits speculative work, which starts before a segment is escalated. The controller begins it whenever it judges an escalation to be near. This mechanism is developed in Section 3.4.4.

A segment reaches the expensive stage when its utility exceeds the price. There the Evidence Grounder (Section 3.5) extracts a structured list of the entities present, each with a short description and its measured motion, and the Scene Memory (Section 3.6) retrieves entries recorded at the same camera. Both are passed to the Reasoner (Section 3.7), a vision-language model used without any fine-tuning. It

returns a verdict, a confidence, and an explanation in which every claim must cite an entry of the evidence list.

The Fusion layer (Section 3.8) enforces the latency contract. It emits a calibrated probability at Δ_{hard} for every segment, from the Sentinel’s output alone. For an escalated segment that value is provisional. If the Reasoner returns in time, the alert is amended in place with a revised probability and its explanation. The Reasoner therefore never sits on the critical path of an alert. The backlog of pending escalations also feeds back into the price. A growing queue raises λ_t , so fewer segments are escalated. The system therefore degrades by explaining fewer events, not by explaining them late.

Figure 3.1 shows the arrangement.

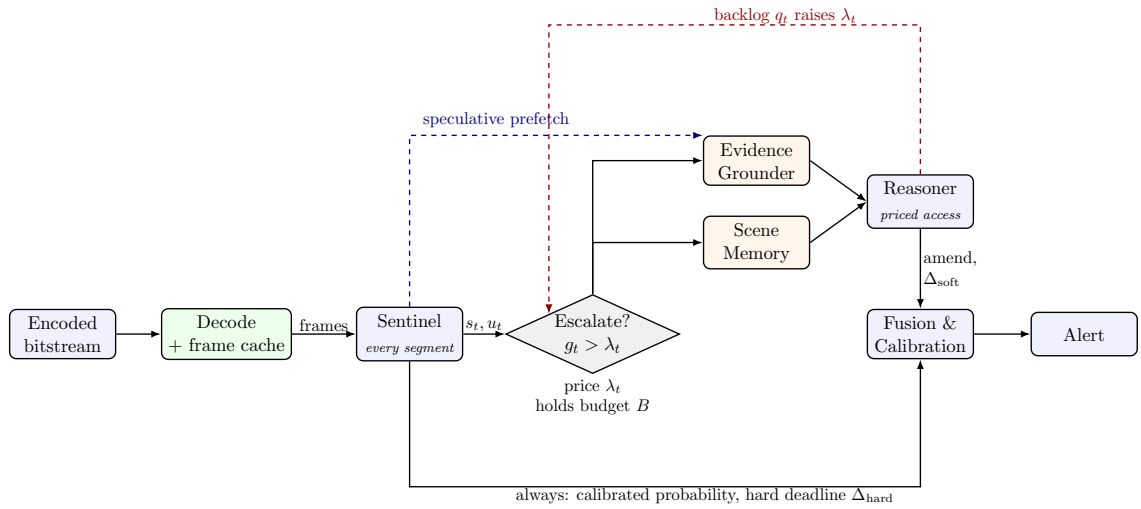


FIGURE 3.1: The ARGUS pipeline, comprising two computational stages of very different cost. Frames are embedded once on arrival and cached, so that the frames shared by overlapping segments are not processed twice. The Sentinel, the cheap stage, scores every segment. The Escalation Controller prices access to the Reasoner, the expensive stage, with a multiplier adapted online to hold a compute budget, and admits speculative work under the same price. Fusion emits a calibrated probability on a hard deadline from the Sentinel alone, then amends the alert when the Reasoner returns. Backlog raises the price, so that overload reduces the number of explanations rather than delaying the score.

3.2.1 Operation on unseen kinds of anomaly

A benchmark contains a fixed set of anomaly types, and a detector can be built to recognize only those. A deployed camera offers no such guarantee. The event that matters may belong to no category anticipated when the system was constructed.

The Sentinel is trained to separate anomalous from normal video, not to name what it has observed, so an unfamiliar event is not excluded by the form of its output. The change-point statistic of Section 3.4 measures each score against what that camera usually sees, not against any training corpus. An event that is ordinary at one camera may therefore be unusual at another.

A segment may escalate for either of two reasons, and it is the second that matters here. It may escalate because its score has risen persistently, or because the evidential uncertainty u_t is high, which occurs when the Sentinel has little evidence for either class. An input unlike anything in training is exactly such a case. It is therefore escalated, rather than given a confident score that happens to be wrong. A softmax would instead pick the closest class and give no sign that it had nothing to go on [21].

The expensive stage is not limited to a fixed set either. The Grounder names objects with an open vocabulary, so a flame or a weapon can be named whether or not it was foreseen. The Scene Memory supplies what is ordinary at the particular camera, which is the reference against which an unfamiliar event is judged unusual. The Reasoner is a general-purpose model, and its verdict is constrained by the evidence it must cite rather than by a list of permitted conclusions. These routes are independent, so an event missed by one may still be admitted by another.

The argument has two limits. The Sentinel learns what counts as normal from its training data, so an event resembling neither the normal nor the anomalous material there may receive a low score. Only the uncertainty route can then escalate it.

The second limit matters more. This section argues that the design leaves a path open for an unfamiliar event. How often that path succeeds is not measured. The benchmarks of Section 5.1 list their anomalies in advance, so none can supply an event outside its own list.

The remainder of this chapter specifies each component in the order data passes through it. It begins with the Sentinel, which scores every segment, and ends with the Fusion layer, which emits the alert. Section 3.9 then analyzes the cost and latency of the assembled pipeline, and Section 3.10 states the complete per-segment procedure.

3.3 The Sentinel

The Sentinel is the only learned component that runs on every segment. Its cost is therefore what the system spends when nothing is happening. Anything it misses is lost, since only the segments it ranks highly reach the expensive stage.

3.3.1 Architecture

The Sentinel embeds each *frame* with a compact backbone and passes the resulting sequence of embeddings to a causal temporal head. The backbone is a two-dimensional convolutional network of at most 20 M parameters (Table 4.1), operating on frames resized to a fixed short side. It is initialized from publicly available ImageNet weights [14] rather than trained from scratch. The weak supervision available here provides too little signal to learn general visual features from random initialization, and a pretrained initialization is standard practice in this setting. The whole network is then fine-tuned end to end under the objective of Section 3.3.3.

The temporal head is a unidirectional recurrent unit [12] or a stack of causal one-dimensional convolutions [2], deliberately not a bidirectional or whole-video transformer. Per-segment inference cost is therefore constant, and independent of how long the stream has been running. A detector operating at a single temporal scale cannot serve both sudden and slow-developing events well. A fight is over in seconds, while an unattended bag becomes suspicious only after minutes of stillness. The two need different amounts of context. We therefore run two such heads in parallel over rolling buffers of different length, a short buffer of T_s segments and a

long buffer of $T_\ell \gg T_s$. Their hidden states are merged by a single lightweight cross-attention layer [46] before the scoring head. Both heads read the same embeddings, so the second scale costs one attention layer rather than a second pass through the backbone. Multi-scale temporal modeling is established practice in weakly supervised anomaly detection [44, 49], and is adopted here rather than introduced. Both buffers are causal. The usual offline formulation is not.

3.3.2 The frame cache

Embedding frames rather than segments is a deliberate choice, and its justification follows from the overlapping segmentation of Section 3.1. With a hop of $L/2$, every frame belongs to exactly two consecutive segments, so a backbone applied to a segment as a whole processes each frame twice. We instead embed each frame once as it is decoded and retain the result in a ring buffer of L embeddings. At each hop, $L/2$ of those are carried over from the preceding segment and $L/2$ are newly computed. The duplication is removed exactly, and the scores produced are unchanged, because the same computation is performed and merely not repeated.

Only the backbone becomes cheaper, not the Sentinel as a whole. Both temporal heads still run once per segment. The Sentinel’s cost divides into a backbone share c_{bb} and a head share c_{head} . The per-segment cost then falls from $c_{\text{bb}} + c_{\text{head}}$ to $\frac{1}{2}c_{\text{bb}} + c_{\text{head}}$. Since $c_{\text{bb}} \gg c_{\text{head}}$ for any convolutional backbone paired with a recurrent head, the always-on cost is close to halved. Section 3.9 writes the factor as one half of c_{sen} .

A second consequence concerns latency rather than cost. A decoder emits frames one at a time, so per-frame embedding spreads the backbone’s work across the time the segment takes to arrive. When the final frame arrives, only the temporal heads, the cross-attention merge, and the scoring head are still to run. All three operate on embeddings rather than on pixels. A backbone applied to the assembled segment would instead perform its entire computation after that last frame, placing it inside the hard deadline of Section 3.8 rather than ahead of it.

The frame cache and the rolling buffers hold different things. The cache holds L *frame* embeddings and exists to avoid recomputation. The buffers hold the recurrent state of the temporal heads over the last T_s and T_ℓ *segments*, and exist to give the two heads different amounts of context. Because the heads are recurrent, neither retains the frames themselves, so a long buffer costs a hidden state rather than $T_\ell L$ embeddings.

The design gives up joint spatiotemporal filtering in the backbone and recovers it only in the temporal head. Section 5.1 compares it against a segment-level three-dimensional backbone [9, 45], so the cost of that choice is measured rather than assumed.

3.3.3 Training objective

The Sentinel must score every segment, but training supplies only one label for each video. A video marked anomalous contains an anomaly somewhere, and nothing says where. The objective closes that gap with three terms. A ranking term uses the video label directly, a pseudo-label term supplies a per-segment signal, and a consistency term decides how far those pseudo-labels can be trusted.

The **ranking term** follows the multiple-instance formulation introduced for this task [44]. Each video is one bag of segments, and the term pushes the maximum score in an anomalous bag above the maximum in a normal bag, by a margin:

$$\mathcal{L}_{\text{rank}} = \max \left(0, 1 - \max_{t \in \mathcal{B}^+} s_t + \max_{t \in \mathcal{B}^-} s_t \right), \quad (3.2)$$

with $\mathcal{B}^+$ and $\mathcal{B}^-$ an anomalous and a normal bag. We retain the standard smoothness and sparsity regularizers on the score sequence [44].

The **pseudo-label term** $\mathcal{L}_{\text{pl}}$ supplies the segment-level signal that Eq. 3.2 lacks. Segments whose embedding lies far from a maintained bank of normal-segment features are treated as likely positives. The idea is borrowed from the nearest-neighbor evidence used by a recent detector to train end to end without segment

labels [23]. The resulting pseudo-label is $\tilde{y}_t \in \{0, 1\}$. The term $\mathcal{L}_{\text{pl}}$ then scores s_t against $\tilde{y}_t$ with a binary cross-entropy, each segment carrying the weight μ_t defined below.

The **consistency term** exists because those pseudo-labels fail in a particular way. Surveillance corpora contain artifacts that look nothing like ordinary footage without being anomalies, such as static title cards, compression glitches and abrupt camera changes. The rule above marks every one of them as a likely positive. The usual remedy is a filter written for one dataset, which does not carry over to another. We instead score each segment twice, under two augmentations. One shifts the segment in time, the other crops it in space. The two predicted distributions over $\{\text{normal}, \text{anomalous}\}$ are written $p_t^{(1)}$ and $p_t^{(2)}$.

Their disagreement does two different jobs. It enters the objective directly as a penalty,

$$\mathcal{L}_{\text{cons}} = \frac{1}{2} [\text{KL}(p_t^{(1)} \| p_t^{(2)}) + \text{KL}(p_t^{(2)} \| p_t^{(1)})], \quad (3.3)$$

which trains the network toward predictions that survive both augmentations. It also sets the weight $\mu_t = \exp(-\mathcal{L}_{\text{cons},t})$ on the pseudo-label term above. A pseudo-label is therefore trusted in proportion to how closely the two views agree. A genuine event is visible under both augmentations. An artifact confined to a single frame typically is not. Consistency regularization of this kind is standard in semi-supervised learning [43], and is used here without modification. It removes a hand-tuned preprocessing step rather than introducing a new way of learning.

The three terms combine as $\mathcal{L} = \mathcal{L}_{\text{rank}} + \beta_1 \mathcal{L}_{\text{pl}} + \beta_2 \mathcal{L}_{\text{cons}}$. The weights β_1 and β_2 are chosen on a held-out validation split.

3.3.4 Evidential uncertainty

The scoring head follows evidential deep learning [42]. Instead of a single logistic output, it parameterizes a Dirichlet distribution over the $K = 2$ classes, normal and anomalous. The head emits non-negative evidence $e_0, e_1 \geq 0$, from which the

concentration parameters are $\alpha_i = e_i + 1$. Their sum is $S = \alpha_0 + \alpha_1 = K + e_0 + e_1$, and one forward pass gives both

$$s_t = \frac{\alpha_1}{S}, \quad u_t = \frac{K}{S}, \quad (3.4)$$

the expected anomaly probability and an uncertainty. Because $S \geq K$ by construction, u_t lies in $(0, 1]$ and reaches one exactly when the network has no evidence for either class. That is the behavior the controller of Section 3.4 relies on. The distinction from a softmax matters here. A softmax can be confidently wrong on inputs unlike anything in training [21]. For a deployed surveillance system that is the common case, not the exception. No sampling is needed, so the head costs only a few output units more than a standard one.

The head itself is adopted unmodified. Only its use is new. The uncertainty u_t enters the escalation decision of Section 3.4, where it must pay the same price as any other signal. “The detector does not know” then becomes a reason to spend budget, rather than a line written to a log.

3.4 The Escalation Controller

The controller decides which segments reach the expensive stage. It therefore sets both how accurate the cascade can be and how much compute it uses. Existing cascaded detectors make this decision with a threshold on a score or an entropy value [15, 27, 51]. A threshold has a defect that tuning cannot repair. It fixes a decision boundary, but what a deployment can afford is a rate. It is calibrated on a validation set to escalate the budgeted fraction B of the segments. On a live camera whose scores are distributed differently, it escalates some other fraction. Nothing reveals the true fraction until the accelerator saturates. Distribution shift is precisely the case a threshold cannot handle. The controller specified below therefore fixes the rate and allows the boundary to follow from it.

3.4.1 Change-point statistic and escalation utility

Two different signals can make a segment worth escalating. The first is evidence that something has changed. We maintain a one-sided cumulative-sum statistic [3, 37] over the Sentinel’s scores,

$$D_t = \max(0, D_{t-1} + (s_t - \bar{s}_t - \delta)), \quad D_0 = 0, \quad (3.5)$$

where $\bar{s}_t$ is a slowly updated estimate of the baseline score level. The slack δ sets the smallest shift worth detecting. The statistic D_t stays near zero while scores fluctuate about their baseline, and grows only when they stay elevated. That makes it far less sensitive to single-segment noise than thresholding s_t directly. The baseline $\bar{s}_t$ is updated only on non-escalated segments, so a developing anomaly cannot raise the level it is judged against.

The second signal is the uncertainty u_t of Eq. 3.4. A segment the Sentinel cannot judge is worth escalating even when nothing has changed. We combine the two into a scalar escalation utility

$$g_t = \omega \tilde{D}_t + (1 - \omega)u_t \in (0, 1), \quad (3.6)$$

where $\tilde{D}_t = 1 - e^{-D_t/\sigma_D}$ squashes the statistic to the unit interval and the weight $\omega \in (0, 1)$ is fitted once on validation data. The interval is open because a weight at either endpoint would discard one of the two signals. It would also allow $g_t = 0$ on a segment whose statistic has not moved.

Comparing g_t against a price is a deferral rule, in the sense of selective prediction and learning to defer [13, 16, 33, 35]. That literature fixes the rule once on held-out data. The controller below instead adapts it online, for the reason given at the start of this section.

The controller does not optimize Eq. 3.1 directly, because that objective is a loss it cannot evaluate online. The label y_t is unavailable at inference. It can instead estimate how far that loss would fall if segment t went to the expensive stage, and

g_t is a surrogate for that. The two signals above mark the cases in which escalation reduces the loss most, which is why the utility is built from them. Fitting ω on validation data ties the surrogate to the loss, since ω is chosen to maximize the loss reduction obtained per escalation. The controller below therefore maximizes $\sum_t a_t g_t$ in place of minimizing Eq. 3.1 under the same budget. Section 5.1 tests that substitution by reporting detection accuracy against the escalation rate, rather than assuming the surrogate is faithful.

3.4.2 Escalation as a constrained decision problem

Both the utility collected and the compute charged are long-run averages, so the decision at one segment cannot be made in isolation from the rest. Let $a_t \in \{0, 1\}$ be the escalation decision and $\hat{a}_t \in [0, 1]$ the accelerator compute that decision charges, defined in Eq. 3.9. The utility g_t is that of Eq. 3.6. The two quantities differ only because the controller also runs speculative work, and are equal when it does not. The controller solves

$$\max_{\{a_t\}} \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T a_t g_t \quad \text{subject to} \quad \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \hat{a}_t \leq B. \quad (3.7)$$

The constraint is stated on charged compute rather than on escalations, because it is the accelerator that is scarce, and every unit it spends is the same resource whether or not the result is used. The objective, however, credits only escalations. A speculation earns no detection utility of its own. A speculation that is never confirmed changes no decision. What it buys instead is a shorter delay before an explanation exists, and Eq. 3.1 accounts for that through its latency constraint rather than through the utility. The asymmetry between the two sides is therefore deliberate: speculation is charged where it consumes and credited where it contributes. Relaxing the constraint with a multiplier $\lambda \geq 0$ [5] makes the per-segment problem separable, and for fixed λ the optimal policy is a comparison against it. The rule below is written in terms of $\tilde{\lambda}_t$, the price the deployed system actually applies. It equals λ_t except under the

congestion of Section 3.4.3, which the analysis that follows sets aside:

$$a_t = \mathbb{I}[g_t > \tilde{\lambda}_t], \quad \tilde{\lambda}_t = \lambda_t \text{ absent congestion.} \quad (3.8)$$

The multiplier is not tuned. It is learned online by projected dual ascent, the standard device for enforcing time-average constraints in stochastic systems [36]:

$$\lambda_{t+1} = [\lambda_t + \eta(\hat{a}_t - B)]_+, \quad \hat{a}_t = a_t + \nu z_t. \quad (3.9)$$

Here $\eta > 0$ is the step size, ν the cost share defined in Eq. 3.10, and $\hat{a}_t$ the *charged* compute rather than the escalation decision alone. Its two terms account for the two ways the accelerator is occupied. An escalation is charged in full. A speculation is charged only if it is wasted. The indicator $z_t \in \{0, 1\}$ records that the speculation started at segment $t - H$ was *not* followed by an escalation within its horizon. Its value is therefore known at time t , even though the speculation itself began earlier. Wasted speculation is charged at

$$\nu = \frac{c_{\text{gr}}}{c_{\text{gr}} + c_{\text{rea}}}, \quad (3.10)$$

where c_{gr} and c_{rea} are the per-segment costs of the Grounder and the Reasoner. A speculation runs the first but never the second, so ν is the share of an escalation's cost that it actually incurs.

Charging on the outcome rather than at the moment of admission is what keeps the accounting exact. A confirmed speculation costs no more than the escalation it precedes, because the Grounder's output is reused rather than recomputed. Charging it separately would bill the same compute twice, hold the realized rate below B , and leave the accelerator idle. Conversely, omitting the wasted case entirely would let the controller hold B on escalations while the accelerator spent strictly more than B . The delay of H segments before z_t is resolved does not affect the argument, since λ_t responds to a time average rather than to individual segments.

The resulting behavior is easy to read. Every escalation raises the price of the next one, and every segment that passes cheaply lets the price decay. On a busy stream the price rises until only the most informative segments can afford the expensive stage. On a quiet stream it falls until the controller is again willing to investigate faint signals. The multiplier λ_t is in this sense a shadow price for accelerator access [24].

The convergence properties of this update are standard, but they hold only under the assumptions set out below. The utility is bounded: $g_t \in (0, 1)$ by construction, as a convex combination of $\tilde{D}_t \in [0, 1]$ and $u_t \in (0, 1]$. The charged compute is bounded too, $\hat{a}_t \in [0, 1]$, because $\nu \leq 1$ and because a_t and z_t cannot both be non-zero for the same unit of compute. The stream must in addition be stationary or ergodic over the horizon considered, and some policy must keep consumption strictly below the budget, a Slater-type condition [5] met here by the policy that escalates nothing. The drift-plus-penalty analysis of Neely [36] then applies: with $\eta = \Theta(1/\sqrt{T})$, both the time-averaged constraint violation and the gap to the best admissible policy vanish at $O(1/\sqrt{T})$.

The guarantee comes with two caveats. Surveillance streams are not stationary over long horizons, so the result describes a stretch over which conditions stay steady, rather than a whole deployment. The speculative term also makes $\hat{a}_t$ depend on λ_t through Eq. 3.15, a feedback the standard analysis does not model. Budget tracking is therefore treated as a property to be measured rather than assumed, and Section 5.1 reports the realized rate against B over time on a dataset the controller was not tuned on.

Two consequences matter in practice. First, λ_t is measured in the same units as g_t and can be logged as a direct measure of how hard the system is working. A rising price is a compact summary of a busy scene. Second, moving to a new camera or dataset requires restating B from the new hardware and nothing else. There is no threshold to re-tune.

The time the controller needs to re-converge follows from the step size. With $\eta = 0.02$ and $B = 0.1$, an escalated segment moves λ by $\eta(1 - B)$ and a segment that is not escalated moves it by ηB . The price therefore climbs across $[0, 1]$ in about

$1/[\eta(1 - B)] \approx 56$ segments and falls across it in about $1/(\eta B) = 500$. The slower direction governs, giving a settling time of a few hundred segments, or two to three minutes of video at the segment rate of Section 3.1. The asymmetry acts in the useful direction, since the price rises quickly once the budget is being exceeded and relaxes slowly when it is not. The slack of Section 3.4.7 is estimated over the same stream and settles more slowly still. That estimate, not the price, therefore governs how long a move to a different scene takes. Both settling times are measured rather than predicted.

3.4.3 Coupling the price to the queue

The budget is a constraint on the average, not on any single moment. A burst of escalations can therefore fill the queue that serves the expensive stage, and a full queue breaks the latency contract of Section 3.8. We add a term that reacts to the instantaneous backlog q_t :

$$\tilde{\lambda}_t = \lambda_t + \kappa[q_t - q^*]_+, \quad (3.11)$$

which raises the price sharply once the backlog exceeds a small target q^* , in the manner of congestion pricing [24]. The effective price $\tilde{\lambda}_t$ is what Eqs. 3.8 and 3.15 compare against, so that admission responds to congestion, while Eq. 3.9 continues to update λ_t alone. Comparing against $\tilde{\lambda}_t$ rather than λ_t is a departure from the relaxation that produced Eq. 3.8, and it is deliberate. The relaxation ignores how escalations are spread over time, whereas a deadline depends on it, so the surcharge trades a small loss of optimality for a bounded queue. Keeping the surcharge outside λ_t matters just as much. It is a transient response to congestion and must disappear once the queue drains. The multiplier λ_t carries the long-run budget, and would decay too slowly if it absorbed a term driven by backlog. This converts the controller from a score threshold into an admission-control mechanism, in the sense familiar from real-time and networked systems [8]. It is also why the system degrades by explaining fewer events rather than by explaining events late.

3.4.4 Speculative escalation

The mechanisms so far control how often the expensive stage runs, not how long it takes. That duration dominates the delay between an event beginning and its explanation appearing, so we begin that stage before the trigger fires.

The idea comes from speculative execution, and in particular from its recent use in speculative decoding for language models [28]. There, a cheap model’s prediction lets expensive work begin early. We apply the same pattern to whole pipeline stages rather than to tokens.

The prediction available here comes from the statistic the controller already maintains. Let

$$\hat{g}_t = \frac{D_t - D_{t-m}}{m} \quad (3.12)$$

be the drift of the change-point statistic over the preceding m segments. Extrapolating that drift projects the statistic to reach $D_t + H\hat{g}_t$ after a horizon of H segments, and escalation occurs when the resulting utility exceeds the price. Let

$$g(D) = \omega(1 - e^{-D/\sigma_D}) + (1 - \omega) u_t \quad (3.13)$$

denote the utility Eq. 3.6 would assign at statistic value D , with u_t held at its present value since the uncertainty of a future segment cannot be known. We then take

$$\hat{\pi}_t = \sigma\left(\beta\left[g([D_t + H\hat{g}_t]_+) - \lambda_t\right]\right), \quad (3.14)$$

a logistic function of the projected margin, with β setting how sharply the estimate saturates. The projection is clamped at zero by $[\cdot]_+$, as the statistic itself is in Eq. 3.5. A falling drift would otherwise carry the projection below the floor that D_t can actually reach, and predict a crossing from a state the process cannot occupy. A hard extrapolation, recovered as $\beta \rightarrow \infty$, would make speculation an all-or-nothing decision on a noisy slope. The smooth form instead lets a marginal projection buy only a correspondingly small claim on the budget. Speculation is then admitted at

the same price, discounted by its probability of being useful:

$$a_t^{\text{spec}} = \mathbb{I}[\hat{\pi}_t g_t > \tilde{\lambda}_t]. \quad (3.15)$$

Speculation is not an additional consumer competing for resources outside the budget. It is priced by the same multiplier, so a speculation unlikely to pay off cannot afford the accelerator. That coupling is what bounds its cost. The wasted fraction of the budget is $\rho_{\text{spec}}(1 - \bar{\pi})$, which we measure directly rather than assume small.

When speculation is admitted, the Grounder begins extraction and the Reasoner's prompt prefix is prepared while the event is still developing. If the trigger fires, only the final generation remains; if it does not, the partial work is discarded at the next frame boundary.

3.4.5 Adaptive window length

The drift $\hat{g}_t$ of Eq. 3.12 serves a second purpose once escalation has been triggered. It shows not only that the statistic is rising but how quickly, and the two cases call for different amounts of context. A sharp rise indicates a brief, sudden event, fully contained in a short window. A gradual rise indicates a slow-developing situation, such as loitering before a theft, whose interpretation needs the preceding minute rather than the preceding second.

The window width w_t is therefore selected from a small ordered set $\mathcal{W} = \{w^{(1)} < w^{(2)} < \dots\}$. The drift is compared against thresholds placed at quantiles of its distribution during a calibration period:

$$w_t = w^{(j)} \quad \text{where } j \text{ is the index with } \hat{g}_t \in [\Gamma_j, \Gamma_{j-1}), \quad (3.16)$$

with $\Gamma_0 = \infty$ and the remaining thresholds decreasing to $\Gamma_{|\mathcal{W}|} = -\infty$. Faster drifts therefore map to shorter windows, and every drift, however small, selects exactly one width. Because they are quantiles rather than absolute values, the same mapping

works on a camera whose scores move slowly and on one whose scores move quickly. Section 3.5 treats speeds the same way.

This avoids the fixed decision period that prior real-time detectors apply regardless of event duration [23]. Because w_t also determines the Reasoner’s input size, it is clipped by the remaining latency budget, as described in Section 3.9.

3.4.6 Arbitrating one accelerator across many cameras

The formulation has so far been written for one camera. Deployments, however, commonly attach several cameras to a single accelerator, and the evaluation protocols used in the surveyed literature [15, 27, 51, 53, 58] do not address this case. The mechanism extends to N cameras sharing one accelerator without modification. Each camera computes its own utility $g_t^{(i)}$ and compares it against a common price, and the dual update aggregates over the fleet:

$$\lambda_{t+1} = \left[\lambda_t + \eta \left(\frac{1}{N} \sum_{i=1}^N \hat{a}_t^{(i)} - B \right) \right]_+. \quad (3.17)$$

The resulting allocation resembles a market [24]. A camera watching an empty corridor cannot outbid a camera watching a disturbance, so accelerator time flows to wherever the evidence currently is, without a central scheduler and without per-camera configuration.

As it stands, however, the arrangement lets a busy camera crowd out a quiet one indefinitely. The quiet camera may be the one that eventually matters. Each camera therefore holds a reservation $B_{\min} \ll B$, implemented as a token bucket of the kind used in real-time servers [8], refilled at rate $B_{\min}$. A token lets a segment be escalated even when $g_t^{(i)} \leq \tilde{\lambda}_t$. Tokens accumulate up to a small cap while a camera is idle, so that a long-silent camera can still escalate when something finally occurs. Reserved escalations are charged to $\hat{a}_t^{(i)}$ exactly as ordinary ones are, so the fleet still respects B in aggregate. The reservation moves budget between cameras without enlarging

it. That requires $NB_{\min} < B$, which bounds the number of cameras one accelerator can serve while still honoring every reservation.

The same mechanism covers a deployment holding more than one accelerator. The controller observes the aggregate escalation rate, the target B , and the backlog, and none of the three depends on how many machines supply the service. A pool of M identical accelerators therefore changes nothing but B , which rises in proportion to the capacity added. Where the machines form separate pools instead of one, whether divided by site or by hardware, the mechanism is replicated rather than extended. One price can balance demand against one supply, so each pool carries its own price, its own budget and its own backlog. Which pool serves a given stream is a load-balancing question. This mechanism neither assigns streams to pools nor depends on the assignment.

3.4.7 Setting the controller's constants

The controller introduces several constants, which divide into those determined by the deployment and those that must be chosen. Only the budget B belongs to the first group. It follows from the accelerator's throughput and the aggregate segment rate, as set out in Section 3.1, and is restated rather than tuned when the system moves to new hardware. The purpose of each remaining constant was given where it was introduced. What follows is how its value is chosen. All are fixed once on a calibration stream held out from both training and evaluation.

The constants shaping the escalation utility are set from statistics of that stream, in the manner standard for change-point detectors [3]. The slack δ must sit above the spread of s_t during quiet periods, so that ordinary fluctuation does not accumulate in D_t . That spread differs between cameras, so δ is set not as a fixed number but as a multiple of the standard deviation estimated at the camera. That estimate is updated alongside the baseline $\bar{s}_t$, from the same non-escalated segments. A fixed number would satisfy the requirement at one installation and fail at another, and the failure is not harmless. Below the spread, D_t accumulates without returning to

zero, which saturates $\tilde{D}_t$ and destroys the event boundary Section 3.9.4 depends on. The scale σ_D is chosen so that a run of segments long enough to constitute an event, rather than a single elevated segment, drives $\tilde{D}_t$ toward its upper range. The weight ω is the exception in this group, being fitted against detection quality on validation data rather than set from a statistic.

The constants governing the controller's dynamics are set from its observed behavior, and the criterion differs for each. The step size η trades convergence speed against steady-state oscillation in λ_t . If η is too small, the budget is tracked sluggishly after a change in scene activity. If it is too large, the price oscillates enough to admit and refuse comparable segments in consecutive intervals. The value chosen is the largest for which the realized rate settles without visible oscillation. The queue target q^* is bounded by the soft deadline rather than chosen freely. A window queued behind q^* others waits for all of them to be served before its own service begins, so it finishes after $q^* + 1$ service times. Admissibility under Δ_{soft} therefore requires

$$(q^* + 1)t_{\text{serve}} \leq \Delta_{\text{soft}}, \quad \text{that is} \quad q^* \leq \frac{\Delta_{\text{soft}}}{t_{\text{serve}}} - 1, \quad (3.18)$$

with t_{serve} the time one escalated window occupies the expensive stage. A standing backlog is thus admissible only when $t_{\text{serve}} \leq \Delta_{\text{soft}}/2$. With $\Delta_{\text{soft}} = 3 \text{ s}$ (Table 4.1), that would require one Grounder and Reasoner pass over a window to finish within 1.5 s. A 7B-scale model over four to eight keyframes is not expected to do so on the target hardware, so the right-hand side of Eq. 3.18 falls below one and q^* is set to zero, which leaves the surcharge to act on the first waiting window. Should the measured t_{serve} prove smaller, Eq. 3.18 gives the admissible target directly rather than requiring a new choice. The gain κ is then the smallest value that returns the queue to q^* within a few segments once the target is exceeded. Both follow from the serving path rather than from the video. By construction, $\hat{\pi}_t = 0.5$ when the projection reaches the price exactly. The sharpness β sets how quickly the estimate moves away from that value, and is calibrated against the observed variability of $\hat{g}_t$.

The reservation B_{min} stands outside this procedure altogether. It is fixed by the requirement $NB_{\text{min}} < B$, together with the longest interval a camera may acceptably

wait before it can escalate on its own.

This procedure has two properties worth stating. First, none of these constants is re-tuned per dataset or per camera, which is what distinguishes the controller from a threshold. A deployment restates B and leaves the rest unchanged. Second, their values matter less than the budget does, since they affect how quickly the constraint of Eq. 3.7 is approached rather than whether it holds. Section 5.1 reports budget tracking on a dataset the controller was not calibrated on, which is the test of both claims.

3.5 Evidence Grounder

A segment whose utility exceeds the price enters the expensive stage. The first task there is to establish what is actually present in the escalated window, before any language model is consulted. This ordering is deliberate. The central trust problem with language-model explanation is that fluency says nothing about correspondence. A model can produce a fluent sentence about a video without describing the video it was actually shown [30, 40]. The Grounder gives the Reasoner something definite to describe, and gives the system an independent list against which that description can be checked.

3.5.1 Extraction

On escalated windows only, we run an open-vocabulary detector [11] followed by a short-horizon tracker [59]. Open vocabulary matters here because the detector is not restricted to a label set chosen at training time, and surveillance anomalies routinely involve objects that no closed vocabulary anticipated.

The detector proposes boxes independently on each sampled frame. The tracker associates them across frames, so that a given person or object receives one identifier and one trajectory for the duration of the window. Everything the Grounder reports about motion is computed from that trajectory, and from nothing else. With (x_i, y_i)

the box center of an entity at sample i and τ the interval between samples, the attributes are

$$\text{speed}_i = \frac{\|(x_i, y_i) - (x_{i-1}, y_{i-1})\|}{\tau}, \quad \text{dwell} = \tau \cdot |\{i : \text{speed}_i < v_{\min}\}|, \quad (3.19)$$

together with the mean and maximum of speed_i over the window and the largest change in heading between consecutive steps, which flags an abrupt change of direction. Here $v_{\min}$ is the speed below which an entity counts as at rest, fixed as described below. Neither optical flow nor a second network is involved, and the cost of these attributes is a few arithmetic operations per entity per frame.

The short descriptive term attached to each entity is not inferred by a second model, but read from these quantities by a fixed rule. An entity is reported as stationary if its mean speed falls below $v_{\min}$, as running if its peak speed exceeds a high percentile of the camera’s distribution, and as moving otherwise. Keeping this mapping fixed and explicit is what allows the resulting list to serve as a reference for checking the Reasoner, since a description produced by another learned component would require verification of its own.

This construction has two details that are easy to get wrong. The first is the sampling rate, which must be high enough for the tracker to associate the same entity across consecutive samples. It is tracking, not the detector’s accuracy, that sets the floor on this rate. A person crossing the frame in two seconds is straightforward to follow at a few frames per second, whereas a rate too coarse to follow fast motion between samples would break the association and therefore the trajectory. We consequently treat the Grounder’s frame rate as a parameter to be set by tracking quality, and report it with the results rather than fixing it in advance.

The second is the choice of units. Speeds derived from box centers are measured in pixels, and pixel speed depends on an entity’s distance from the camera. A raw value is therefore not comparable between scenes, or even between regions of one scene. Rather than calibrate the camera, which would require information a deployment

cannot be assumed to have, we express each speed as a percentile of the distribution observed at that camera during the ordinary activity already assumed in Section 3.1. An entry therefore asserts that an entity is moving faster than is typical for this camera, which is the claim the Reasoner requires and the claim a percentile can support. The threshold $v_{\min}$ separating motion from rest is set as a low percentile of the same distribution, so that both are expressed on one scale.

Extraction is written to be interruptible, because the Grounder is the component started first when the controller speculates (Section 3.4.4). A speculative extraction that is never confirmed is abandoned at the next frame boundary and carries no state forward. That is what bounds the cost of a wasted speculation by the fraction ν of Eq. 3.10 rather than by the full cost of an escalation.

3.5.2 Structured evidence list

The output is a compact list $\mathcal{E}_t$ of entries such as

```
person#1:  running, peak speed 97th pct, present from window start
backpack#2: stationary, dwell 14s, nearest person 41% of frame width
```

Each field is derived from the trajectory by the rules above, and the entries report no quantity that would require camera calibration. Separations between entities are stated as a fraction of frame width, and speeds as percentiles, because a deployment cannot be assumed to supply the geometry that would convert either into metres. The list is produced without any language-model call, and is therefore cheap, deterministic and reproducible.

These properties are what make the list usable as a reference. Dense captioning of every frame, as used by offline explanation pipelines [56], offers no equivalent guarantee, since there is no fixed procedure for deciding whether a caption’s claim is visible in the frame that produced it. Replacing free text with a list of identifiers converts the question “is this description trustworthy” into “do the cited identifiers exist,” and the latter is answerable by string matching against $\mathcal{E}_t$.

3.5.3 Canonical form

The list is emitted in canonical form: entity labels sorted, motion attributes quantized into coarse bins, identifiers assigned deterministically within a window. The operation costs nothing beyond a sort and serves two purposes. It makes the list hashable, so that consecutive escalations within one event can be recognized as posing the same question to the Reasoner, and the redundant call can be suppressed (Section 3.9). It also makes the list stable across runs, so that the citation check of Section 3.7 is reproducible rather than dependent on the detector’s output order. Quantization is what makes hashing possible: without it, small fluctuations in measured speed would defeat every match.

3.6 Scene Memory

The evidence list records what is present in an escalated window, but not whether its presence is unusual at this particular camera. That judgement requires knowledge of the scene, and it is knowledge no benchmark supplies. Every detector trained on a fixed benchmark carries the assumption that the deployment scene resembles its training distribution. In practice a camera at a loading dock and a camera in a quiet corridor have incompatible notions of normal, since forklifts and pallets move constantly at the first and almost never at the second. Training on public benchmarks [44, 48] cannot anticipate the full range of such scenes. The Scene Memory supplies this knowledge without altering any model weights, in the manner of retrieval-augmented generation [29].

3.6.1 Construction and retrieval

We write $\mathcal{M}$ for the memory, a set of (embedding, verdict) entries. At deployment, a few tens of short clips of ordinary activity at the target camera are embedded with a frozen image-text encoder [39, 57]. Each is added to $\mathcal{M}$ with a normal label, in a small vector index [22]. A clip is represented by the mean of its per-frame

embeddings, which is sufficient for retrieval and avoids introducing a second temporal model. Neither labeling nor fine-tuning is required to seed the memory. The only assumption is the one already stated in Section 3.1, that a short period of ordinary operation can be observed before the system is relied upon.

An escalated window is embedded by the same encoder and in the same way, so that query and stored entries occupy one representation space. Embeddings are ℓ_2 -normalized and compared by cosine similarity, so that a larger value indicates a closer match. The k entries of highest similarity are retrieved from $\mathcal{M}$. An entry is returned only if its similarity to the query exceeds a threshold θ_s . That threshold is set from the distribution of pairwise similarities among the seed clips themselves, so that the bar adapts to how varied ordinary activity at that camera happens to be. The threshold matters because a fixed k would otherwise return the k most similar entries no matter how dissimilar they are. A window unlike anything the camera has recorded would then be described to the Reasoner as resembling ordinary activity.

When no entry survives the threshold, the Reasoner is told that the window resembles nothing in the camera’s record. That statement is itself informative, and it is the correct input for the open-set case discussed in Section 3.2.1.

Surviving entries are rendered as short fixed-form statements rather than passed as vectors, since the Reasoner consumes text. An entry contributes a line of the form *a visually similar window at this camera was previously judged normal*, or the corresponding statement for an anomalous verdict. Each line also carries its rank among the retrieved set. These lines supply the knowledge of the scene that a generically trained model does not have.

3.6.2 Updating without gradients

When the Reasoner returns a verdict v_t on an escalated window, the pair consisting of that verdict and the window’s embedding is added to $\mathcal{M}$ immediately. The write is automatic, forms part of the same asynchronous path as the amendment of Section 3.8, and waits on no external input.

An operator may later review the alert, and that review reaches the memory without disturbing the automatic write just described. Each entry is tagged with the identifier of the alert that produced it, so a review is applied as an update to the existing entry rather than as a new one. The verdict is replaced if the operator disagrees with it, and the entry’s retrieval weight is raised to the level described in Section 3.6.3. No embedding is recomputed, since the stored entry already holds it. The review itself is a binary confirmation recorded on the alert in front of the operator, and demands no judgement they are not already making.

Nothing here is time-critical. Reviews arrive after an unbounded delay, or not at all. Entries carrying operator confirmation are exempt from eviction, so that the most reliable records are also the most durable. If a review arrives for an entry already evicted, the entry is reinserted from the alert record, which retains the embedding for this purpose.

Because a memory write is not a weight update, it carries none of the usual hazards of continual fine-tuning. No previously learned competence is disturbed, since no parameter changes. The update takes effect immediately, without retraining. Any entry can also be pruned or corrected afterwards if it proves wrong. Of the three, the ability to undo matters most. A wrong entry can be removed. Fine-tuning offers no comparable way to take back a change once it is absorbed into the weights. Entries are nonetheless discarded over time under the bound of Section 3.6.3. That discarding is deliberate and confined to individual records, unlike the diffuse loss of prior competence that continual gradient updates risk [25].

Scene Memory is lighter than inducing a fixed rule set once from a handful of examples, as in prior few-shot rule-based detection [52]. It continues to evolve with the deployment instead of freezing after an induction step. It is also evaluated on multi-scene weakly supervised benchmarks rather than on a single controlled scene.

3.6.3 Bounding the memory and its errors

$\mathcal{M}$ is governed by two constraints. The first caps its size at a few hundred entries per camera. Least-recently-used eviction is applied only to entries written during operation, so that the clips seeded at deployment are never displaced by the system’s own subsequent writes. Storage therefore grows with the number of cameras rather than with the length of a deployment. At the embedding widths in use it amounts to well under a megabyte per camera.

The second addresses a failure specific to a memory that records its own judgements. An incorrect verdict, once written, becomes evidence supporting the same judgement on the next similar window. Repeating that cycle would let an error entrench itself. Entries are therefore weighted by the reliability of their source. Clips seeded at deployment and entries confirmed by an operator carry full weight, whereas entries resting on a Reasoner verdict alone carry less. The system’s own output cannot outweigh the record it was given or the corrections it has received.

3.7 Reasoner

The Reasoner is the only large model in the pipeline and the only component producing natural language. It is used entirely frozen: no instruction corpus is built and no weights are updated.

3.7.1 Why the model is frozen

Fine-tuning does not address the failure this work is concerned with. Training a vision-language model on anomaly-specific instructions improves fluency and domain vocabulary, but it does nothing by itself to guarantee that the resulting text describes the frames the model was shown. Grounding does a different job from fine-tuning rather than replacing it, and it is grounding that makes an explanation checkable. A frozen model made reliable through grounding and retrieval also remains easier to

maintain as stronger open models are released, since upgrading means swapping the model rather than retraining it.

Resources point the same way. Building an instruction corpus at the scale used by recent fine-tuned surveillance assistants [53, 58] was beyond this project. Combining grounding with parameter-efficient tuning, once a small labeled explanation set exists, is left to future work.

3.7.2 Prompt construction

For an escalated window the Reasoner receives four inputs: a fixed instruction specifying the output format, the entries $\mathcal{R}_t$ retrieved from Scene Memory, a small number of representative keyframes sampled from the window, and the evidence list $\mathcal{E}_t$.

That order is deliberate. The instruction and the retrieved entries are stable across escalations from a given camera, whereas the keyframes and the evidence list differ on every call. Placing the stable parts strictly first allows their key/value tensors to be retained and reused across calls [26] (Section 3.9). The more natural-reading arrangement puts the evidence first and the instruction last. That would invalidate the cache on every call, and so multiply the cost of the system’s most expensive component.

3.7.3 Contractual output and the citation check

The Reasoner is instructed to return a structured record

$$(v_t, c_t, r_t, \text{cited_ids}), \quad (3.20)$$

with v_t a normal/anomalous verdict, $c_t \in [0, 1]$ a stated confidence, r_t the explanation text, and `cited_ids` a list of entity identifiers drawn from $\mathcal{E}_t$. The confidence is recorded as the model reports it and is recalibrated before use, for the reason given

in Section 3.8. The recalibrated value $\tilde{c}_t$ enters the Fusion layer, never the stated one. The verdict v_t should not be confused with y_t , which is reserved throughout for the ground-truth annotation of Eq. 3.1 and is never available to the system.

Four conditions must hold for a record to be accepted, and a single recovery procedure applies whenever any of them fails. The first is that the record must parse into the four fields. The second is that every entity identifier appearing in the explanation r_t must also appear in `cited_ids`. The third is that every identifier in `cited_ids` must exist in $\mathcal{E}_t$. The fourth is that `cited_ids` must be non-empty whenever $\mathcal{E}_t$ is non-empty and the verdict is anomalous. On failure, one re-prompt is issued with the specific defect stated. If the second attempt also fails, the explanation is suppressed and replaced by a plain rendering of the evidence list, while the provisional probability of Section 3.8 continues unaffected. This converts hallucination from a subjective judgement into an automatic and reproducible check, and Section 5.1 reports the rate at which each condition fires.

The first condition exists because a frozen model is not guaranteed to follow the format it was asked for. The record is parsed rather than assumed. The need to enforce the format at inference, rather than to train the model to produce it, is one cost of keeping the model frozen. The frequency of parse failures is reported alongside the other three conditions.

The second and third conditions constitute the citation check, and are deliberately mechanical. Both are decided by string matching against a list the Grounder produced deterministically, with no interpretation of the explanation and no second model involved. A check that instead had to decide which entities an explanation implicitly depends upon would require a model of its own, and that model would in turn require verification. Restricting the check to identifiers keeps it a fixed procedure rather than another judgement.

The fourth condition closes a gap the second and third leave open. A record citing nothing at all would satisfy both, since a condition on every identifier holds automatically when there are none. An assertion that something is wrong without reference to anything observed would then pass a check designed to catch exactly

that failure. Requiring at least one citation when evidence exists and an anomaly is declared places such a record on the same footing as one citing an entity that was never detected.

The scope of the check is narrow. It verifies that the explanation refers to entities the Grounder actually detected. It does not verify that those detections are correct, nor that the reasoning connecting them is sound. It is a necessary condition for faithfulness [34] rather than a sufficient one, and is reported as such. The human evaluation described in Section 5.1 exists to cover what the automatic check cannot.

3.8 Fusion and the Two-Deadline Contract

The Fusion layer combines the signals available at each deadline into a calibrated probability and decides when that probability must be emitted. The second of these is the one prior cascaded systems leave implicit, and answering it explicitly is what meets the latency constraint of Eq. 3.1.

3.8.1 The two deadlines

In a conventional cascade the alert for an escalated segment is emitted after the large model returns. The consequence is that the system responds fastest when it has least to report and slowest when an event is in progress. The escalated segments, which are precisely the ones an operator needs promptly, are the ones made to wait for a multi-billion-parameter model. The system’s worst-case latency also becomes the slow path’s worst case, which on shared or networked hardware is effectively unbounded.

The system therefore commits to two deadlines. A calibrated anomaly probability is emitted for every segment within Δ_{hard} , whether or not that segment was escalated. An explanation should accompany an escalated segment within Δ_{soft} , after which the alert stands without one. Only the Sentinel and the controller lie inside Δ_{hard} ,

so the hard deadline is met by construction and is independent of the Reasoner, the network, and the queue depth.

The two values are not free parameters, and each is fixed for a different reason. The hard deadline is bounded above by the rate at which segments arrive. With the segmentation of Section 3.1 a segment arrives every 0.32 s. A system that took longer than that to score one would accumulate an unbounded backlog, so any admissible Δ_{hard} must lie below the inter-arrival interval. The value used here sits slightly under it. This is a stability requirement rather than a preference, and it is what makes the hard deadline a property of the design rather than a target to aim at.

The soft deadline is set by the operator rather than by the machine. An explanation is useful only if it arrives before the operator has decided what to do about the alert, which places Δ_{soft} at the scale of a few seconds. One arriving later is a record of what happened rather than an aid to acting on it. The tolerance ϵ on hard-deadline misses is, by contrast, a design choice. A system that tolerated no misses at all would require a known worst-case execution time for the accelerator [8], which general-purpose hardware does not provide. A small violation rate is therefore accepted and measured instead of being assumed away. All three values are restated in Table 4.1 and their attainment is reported in Section 5.1.

3.8.2 Emission and amendment

Every segment is scored at the hard deadline from the signals the Sentinel and the controller have already produced. The combiner is logistic, but its parameters depend on whether the segment was escalated:

$$\hat{p}_t^{\text{prov}} = \begin{cases} \sigma(\theta_1^n s_t + \theta_2^n \tilde{D}_t + \theta_0^n), & a_t = 0, \\ \sigma(\theta_1^e s_t + \theta_2^e \tilde{D}_t + \theta_0^e), & a_t = 1, \end{cases} \quad (3.21)$$

where the θ are fitted coefficients and the superscripts distinguish the non-escalated and escalated cases. A segment is escalated precisely because the controller judged

it worth examining, so escalated segments carry a substantially higher anomaly base rate than the stream at large. A single parameter set fitted across both would be systematically miscalibrated on each. The same argument separates the provisional and amended maps below. For the great majority of segments the first branch is the sole output, and its value is final at the moment it is emitted. A probability is *provisional* only in the second branch, where an amendment is possible.

When the Reasoner returns within Δ_{soft} , the alert is amended rather than replaced. It retains its identifier and original timestamp and acquires a revised probability together with an explanation:

$$\hat{p}_t^+ = \sigma(\psi_1 s_t + \psi_2 \tilde{D}_t + \psi_3 \tilde{c}_t \mathbb{I}[v_t = \text{anomalous}] + \psi_0). \quad (3.22)$$

The coefficients ψ are fitted in the same way, and the indicator includes the Reasoner's confidence only when its verdict is anomalous. The confidence entering this expression is not the value the Reasoner states but a recalibrated form of it. A frozen vision-language model reports confidence that is typically overstated and related to its accuracy non-monotonically. The value $\tilde{c}_t$ is therefore obtained by binning the stated c_t on the held-out split and replacing each bin's value by the empirical accuracy observed within it [19]. A linear coefficient alone could not correct a distortion of that shape, and leaving c_t uncorrected would import the model's overconfidence directly into a probability the system presents as calibrated.

An escalated alert may receive no amendment by Δ_{soft} , whether because the Reasoner was slow, unreachable, or produced a record the contract of Section 3.7 rejected. At that deadline it ceases to be marked provisional and stands on $\hat{p}_t^{\text{prov}}$ alone. No alert therefore remains in an indeterminate state indefinitely.

All three parameter sets are fitted on a held-out split, and each carries its own recalibration map. Collapsing any two of them introduces a systematic bias, because each describes a population with a different anomaly base rate: segments never escalated, segments escalated but not yet reasoned about, and segments for which a verdict has arrived. Calibration is therefore reported separately for each.

Because an amendment can change what the system asserts, two quantities are treated as results in their own right, not as implementation detail. The amendment rate is the fraction of alerts whose probability moves materially after reasoning. The flip rate is the fraction whose verdict crosses the operator’s decision threshold in either direction. A system that frequently retracts is unusable however good its final answers are. The operator-facing display marks an escalated alert as provisional until it is amended or its soft deadline passes. Both rates are reported alongside accuracy.

3.8.3 Calibration as a primary metric

Because $\hat{p}_t^{\text{prov}}$ and $\hat{p}_t^+$ are read by the operator deciding whether to act, their calibration matters as much as their ranking quality. A system reporting ninety percent confidence while being right sixty percent of the time is unsafe to rely on even if it separates normal from anomalous well by area under the curve. Expected calibration error [19] is therefore reported as a primary metric. The mapping to probabilities uses temperature or Platt scaling [38] fitted on held-out data rather than a weighting chosen by inspection.

3.8.4 Failure containment

The contract also simplifies failure handling. If the Reasoner is unreachable, returns late, or twice returns a record failing any condition of Section 3.7, neither a recovery path nor a fallback branch is needed. The provisional alert has already been emitted and is calibrated. It simply never receives an amendment. A failure of the expensive stage costs an explanation, never an alert and never a missed deadline.

3.9 Cost and Latency Analysis

The preceding sections specify what each module computes. This section states what the system costs and how long it takes. Both are modelled explicitly rather than left to be discovered at benchmark time, for two reasons. They identify which term dominates and therefore which optimization is worth building. They also turn claims about cost and latency into predictions the evaluation can falsify.

3.9.1 Cost model

The per-segment costs of decoding, the Sentinel, the Evidence Grounder and the Reasoner are written c_{dec} , c_{sen} , c_{gr} and c_{rea} respectively. Three rates enter alongside them: the escalation rate ρ , the memoization hit rate h defined below, and the speculation rate ρ_{spec} with mean success $\bar{\pi}$. For the long-run average cost per segment $\bar{C}$, taken in the same sense as the averages of Eq. 3.1,

$$\bar{C} = c_{\text{dec}} + \frac{1}{2} c_{\text{sen}} + (\rho + \rho_{\text{spec}}(1 - \bar{\pi}))c_{\text{gr}} + \rho(1 - h) c_{\text{rea}}. \quad (3.23)$$

The third term counts grounding once for every escalation and once more for every speculation that was not confirmed. A confirmed speculation adds nothing, since its extraction is reused by the escalation that follows and is already counted in ρ . This is the same accounting the controller performs in Eq. 3.9, expressed in absolute cost rather than as the fraction ν of an escalation. Charging ν there and c_{gr} here are two statements of one quantity, since Eq. 3.10 defines ν as exactly the grounding share of an escalation's cost.

It follows that ρ is bounded by, rather than equal to, the budget. The controller holds $\hat{a}_t$ at B , and $\hat{a}_t$ includes the charge for wasted speculation, so

$$\rho + \nu \rho_{\text{spec}}(1 - \bar{\pi}) \approx B, \quad (3.24)$$

and the realized escalation rate is strictly below B whenever speculation is active. The gap is the price paid for the lead time that speculation buys. Section 5.1 reports the realized rate and that lead time together, so the exchange can be judged rather than assumed favorable.

A segment is charged when it is admitted. The hash that decides whether the Reasoner runs is computed only afterwards, so the charge cannot depend on it. An escalation later served from the memoization cache therefore pays for a call it never makes. The accelerator is charged $\rho h c_{\text{rea}}$ per segment more than it performs. The direction of the error is conservative, since realized load falls below the budget rather than above it. Section 5.1 also reports realized load alongside B , so the size of that gap is visible.

The charge could be made exact by crediting a hit through the same outcome-based route that Eq. 3.9 uses for wasted speculation. That change would carry two costs of its own. The charge would no longer be confined to $[0, 1]$. The price would also fall during an event, which is when hits are most frequent and when a further escalation most often repeats a verdict already held.

The remaining structure of Eq. 3.23 sets the priorities. Since $c_{\text{rea}} \gg c_{\text{sen}}$, the final term dominates unless ρ is small, which is what the budgeted controller guarantees. Memoization reduces that term further by the factor $(1 - h)$. The factor of one half on c_{sen} follows from the frame cache of Section 3.3.2. With a hop of $L/2$ each frame belongs to exactly two segments. Embedding it once rather than once per segment therefore halves the backbone's contribution exactly and without any effect on the scores produced. Applying that factor to the whole of c_{sen} is a slight simplification. The Sentinel's cost decomposes as $c_{\text{sen}} = c_{\text{bb}} + c_{\text{head}}$, with the two terms defined in Section 3.3.2. Only c_{bb} is halved, since the temporal heads run once per segment regardless. The approximation is therefore accurate to the extent that c_{bb} dominates. Section 5.1 reports the two contributions separately, so that extent can be checked. The decode term is written explicitly because it is routinely omitted from this literature [44, 48], where evaluation on pre-extracted features sets it to zero. It is paid on every segment and is measured rather than assumed.

3.9.2 Latency model

The two deadlines have structurally different budgets:

$$L_{\text{hard}} = t_{\text{dec}} + t_{\text{head}} + t_{\text{fuse}}, \quad (3.25)$$

$$L_{\text{soft}} = (1 - h) \left[\max(0, t_{\text{gr}} + t_{\text{prefill}} + t_{\text{gen}} - \Lambda_{\text{spec}}) + t_{\text{queue}} \right]. \quad (3.26)$$

Eq. 3.25 contains no term depending on escalation, which is the defining property of the contract. It is also short. Each frame is embedded as it arrives. The work outstanding when the final frame of a segment has been decoded is therefore that frame’s embedding, one step of each of the two temporal heads, the cross-attention merge and the scoring head. Together these are denoted t_{head} . All but the first operate on embeddings rather than on pixels, and the first is a single frame rather than L of them. A backbone applied to the segment as a whole [9, 45] would instead place its entire cost after the last frame arrives. That is the same computation, but positioned inside the deadline rather than spread across the interval preceding it.

Eq. 3.26 is written as an expectation over escalated segments, in which the factor $(1 - h)$ accounts for those served from the memoization cache of Section 3.9.4. A cache hit returns a stored record and incurs none of the terms in brackets. The remaining mechanisms of this section act on the bracket itself. Speculation subtracts the lead time Λ_{spec} , which cannot exceed the horizon H over which the speculation was admitted and in practice equals the grounding work completed before the trigger fired. Prefix reuse shrinks t_{prefill} , and the price-queue coupling of Eq. 3.11 bounds t_{queue} .

3.9.3 Per-camera prefix cache

The Reasoner’s prompt has a stable part and a volatile part. The cache exploits that split. Recomputing attention over the instruction and the retrieved entries on every escalation is redundant. For a long instruction and a handful of entries, that redundant work can exceed the cost of the tokens that actually differ. The

key/value tensors of the stable prefix are therefore retained on the accelerator that serves the Reasoner, keyed by the camera they were built for. They are reused across escalations of that camera, and invalidated only when its Scene Memory changes.

Prefix reuse is routine in language-model serving [26]. Its effect on this pipeline is nonetheless structural rather than incremental. It converts per-camera context from a recurring per-call cost into a one-off cost, which is what makes carrying scene-specific entries affordable at all. In a fleet deployment the cache’s footprint therefore grows with the number of cameras rather than with stream length, and least-recently-used eviction bounds it.

3.9.4 Evidence-hash memoization

Escalations are not independent. A single event typically triggers several consecutive escalations whose evidence lists are identical up to noise, since the same entities are present and exhibit the same motion attributes throughout. Within one event, submitting the same question to the Reasoner repeatedly pays the dominant term of Eq. 3.23 again, without altering any verdict.

The canonicalized evidence list of Section 3.5.3 is therefore hashed. If the hash matches that of a recent escalation within the same event, the previous verdict, confidence and explanation are reused and no call is made. The alert records that its explanation was carried forward. The hash is constructed so that it must change whenever any entity enters or leaves the scene, which bounds the staleness a carried-forward explanation can acquire. An event whose composition is genuinely evolving cannot be summarized by a stored record. The hit rate h is reported in Section 5.1 rather than assumed.

The event boundary that memoization depends on is taken from the controller rather than introduced as a separate parameter. An event is the maximal run of consecutive segments over which the change-point statistic D_t remains above zero.

3.9.5 Deadline-aware degradation

Under load a real-time system must decide what to surrender. ARGUS degrades along a defined axis rather than by dropping work. That axis is the size of the Reasoner's input, counted in keyframes and evidence entries. The size is set by the time remaining before Δ_{soft} . The expensive stage is thereby an anytime computation [60], whose output quality rises with the budget it is given.

That axis has a floor. With a comfortable budget the Reasoner receives the full window. Under pressure the input contracts to one keyframe and the highest-ranked evidence entries. That is the smallest input from which a citation can still be produced and checked. The contract of Section 3.7 could not be enforced below that point, so the floor is set by the grounding requirement rather than by the schedule.

The alternative arrangement is a fixed input size with requests dropped once the queue fills. That discards whole events rather than detail, which is the worse failure for an operator. The price-queue coupling of Section 3.4.3 provides a second response, shedding load before the queue deepens. The system therefore has two graceful responses to overload before it must refuse work.

3.10 The Complete Procedure

Each component has so far been specified in isolation. Algorithm 1 assembles them, stating in full what the system does when one segment arrives. It is written to be read alongside Figure 3.1. The steps follow the same path through the two stages, with the equations that govern each decision cited at the point they are applied.

Two properties of Algorithm 1 follow from its ordering. The provisional emission occurs before any branch that could block, which is what makes the hard deadline structural rather than best-effort. And the baseline update occurs only in the non-escalated branch, so the reference level against which anomalies are measured cannot drift toward an anomaly in progress.

Input: price λ , budget B , statistic D , baseline $\bar{s}$, memory $\mathcal{M}$, backlog q , unused-speculation flag z

foreach *newly decoded frame f of the segment* **do**

 append $\text{EMBED}(f)$ to the frame cache // each frame embedded once

end

$(s, u) \leftarrow \text{TEMPORALHEADS}(\text{cached embeddings for this segment})$ // Eq. 3.4

$D \leftarrow \max(0, D + s - \bar{s} - \delta); \quad g \leftarrow \omega \tilde{D} + (1 - \omega)u$ // Eqs. 3.5, 3.6

$\tilde{\lambda} \leftarrow \lambda + \kappa [q - q^*]_+$ // Eq. 3.11

$a \leftarrow \mathbb{I}[g > \tilde{\lambda}]$ // Eq. 3.8

if $a = 1$ **then**

$\text{EMITPROVISIONAL}(\sigma(\theta_1^e s + \theta_2^e \tilde{D} + \theta_0^e))$ // hard deadline met here

else

$\text{EMITPROVISIONAL}(\sigma(\theta_1^n s + \theta_2^n \tilde{D} + \theta_0^n))$ // Eq. 3.21

end

$\hat{g} \leftarrow (D - D_{t-m})/m; \quad \hat{\pi} \leftarrow \sigma(\beta[g([D + H\hat{g}]_+) - \tilde{\lambda}])$ // Eqs. 3.12, 3.14

if $\hat{\pi} g > \tilde{\lambda}$ **then**

 BEGINGROUNDING and WARMPREFIX asynchronously // Eq. 3.15

end

if $a = 1$ **then**

$w \leftarrow \text{WINDOWLENGTH}(\hat{g})$ clipped by remaining budget // Eq. 3.16

$\mathcal{E} \leftarrow \text{GROUNDER}(w); \quad \mathcal{R} \leftarrow \text{RETRIEVE}(\mathcal{M}, w, k)$

if $\text{HASH}(\mathcal{E})$ *seen in current event* **then**

$(v, c, r) \leftarrow \text{cached record}$ // Section 3.9.4

else

$(v, c, r) \leftarrow \text{REASONER}(\text{prefix}, \mathcal{R}, \text{keyframes}, \mathcal{E})$

$(v, c, r) \leftarrow \text{CONTRACTCHECK}(v, c, r, \mathcal{E})$ // one re-prompt, else suppress

end

$\tilde{c} \leftarrow \text{RECALIBRATE}(c)$

$\text{AMENDALERT}(\sigma(\psi_1 s + \psi_2 \tilde{D} + \psi_3 \tilde{c} \mathbb{I}[v=\text{anom}] + \psi_0), r)$ // Eq. 3.22

$\mathcal{M} \leftarrow \mathcal{M} \cup \{(\text{EMBED}(w), v)\}$

else

$\bar{s} \leftarrow (1 - \gamma)\bar{s} + \gamma s$ // baseline updated only when not escalated

end

$z \leftarrow \mathbb{I}[\text{speculation admitted at } t - H \text{ was never escalated}]$

$\hat{a} \leftarrow a + \nu z; \quad \lambda \leftarrow [\lambda + \eta(\hat{a} - B)]_+$ // Eq. 3.9

Algorithm 1: ARGUS online processing of one segment

3.11 Summary

This chapter has specified a two-stage cascade governed by a single controller and a single latency contract. The design follows from the two constraints stated in Eq. 3.1 rather than from a preference for any particular model.

The compute constraint produces two mechanisms. The budgeted controller prices access to the expensive stage, so that the compute charged against it tracks a value set by the hardware rather than drifting with the score distribution of whatever camera is connected. The frame cache removes the duplicated work that overlapping segments would otherwise incur in the always-on stage. The latency constraint produces the two-deadline contract. That contract places the calibrated score on a hard deadline met by the streaming tier alone, and allows the explanation to arrive afterwards as an amendment. Speculation, prefix caching and memoization shorten the expensive stage when it does run.

Two commitments cut across the whole design. Calibration is treated as a property to be measured rather than assumed, because the output is read by the operator deciding whether to act. Every explanation is also required to cite evidence obtained independently of the model that wrote it. That makes an ungrounded explanation automatically detectable rather than a matter of opinion.

The configuration and development plan for these components are given in Chapter 4, and the protocol that will test whether the design delivers what it claims is given in Chapter 5.

Chapter 4

Implementation

This chapter records how the design of Chapter 3 is to be realized: which components are trained and which are used off the shelf, the configuration each is given, the order in which the system is built, and the risks that order is arranged to contain.

4.1 Component Choices and Compute Budget

Three components are pretrained: the vision-language Reasoner, the image-text encoder used for Scene Memory embeddings, and the open-vocabulary detector used by the Grounder. Each is an openly released model with public weights. No component depends on a proprietary interface to run end to end. A hosted endpoint remains an available fallback for the Reasoner, if local inference on the target device proves too slow. The only component trained here is the Sentinel. Its backbone is initialized from public ImageNet weights [14] and fine-tuned rather than trained from scratch. Its parameter count and training-set size are modest, so it can be trained on a single consumer or university GPU in one to two days per dataset. The Grounder, the Reasoner, and the Scene Memory encoder are used frozen, which removes any need for an instruction-tuning corpus.

4.2 Configuration

Table 4.1 lists the initial settings. These are design choices and starting points for tuning on validation data, not measured results. They are marked as such because the distinction is easily lost once a table exists.

TABLE 4.1: Initial configuration. All values are starting points to be tuned on a held-out validation split; none is a reported result.

Component	Setting	Initial value
Segmentation	length L / hop	16 frames / 8 frames at 25 fps
Frame cache	size	$L = 16$ embeddings
Sentinel backbone	initialization	ImageNet-pretrained, then fine-tuned
Sentinel backbone	parameters	≤ 20 M
Sentinel backbone	input	short side 256, centre region 224
Sentinel buffers	T_s / T_ℓ	4 / 24 segments (≈ 1.3 s / 7.7 s)
Sentinel head	output	Dirichlet, 2 concentrations
Loss weights	β_1 / β_2	0.5 / 0.2, tuned on validation
Optimization	optimizer / schedule	AdamW, cosine decay, 50 epochs
Budget	B	0.10 (swept in evaluation)
Utility	weight ω	0.7, fit on validation
CUSUM	slack δ	$1.8 \hat{\sigma}_s$, estimated at the camera
CUSUM	baseline rate γ	10^{-2}
Controller gains	η / κ	0.02 / 1.0, from a calibration stream

Table 4.1 (continued)

Component	Setting	Initial value
Queue target	q^*	0, from the bound of Section 3.4.7
Squash and sharpness	σ_D / β	from the same stream (Section 3.4.7)
Speculation	horizon H / lookback m	2 / 5 segments
Speculation	charge share ν	from measured $c_{\text{gr}}, c_{\text{rea}}$
Window lengths	set $\mathcal{W}$	$\{2, 6, 16\}$ segments
Grounder	sampling rate	2 fps within the window
Scene Memory	seed size / retrieved k	32 clips / 4
Scene Memory	similarity threshold θ_s	from seed-clip distribution
Scene Memory	capacity per camera	256 entries, LRU over written entries
Reasoner	class / keyframes	open 7B-scale VLM / 4–8
Deadlines	$\Delta_{\text{hard}} / \Delta_{\text{soft}}$	300 ms / 3 s
Fleet reservation	B_{min}	0.1 B per camera

4.3 Development Plan

The ordering below is chosen so that the highest-risk contribution is validated before anything depends on it.

1. Reproduce a real-time weakly supervised baseline as a sanity check of the Sentinel training pipeline.
2. Add multi-scale fusion and the consistency term; validate against the baseline.

3. Restructure the backbone to embed frames rather than segments and add the frame cache, confirming that scores are unchanged and measuring the reduction in c_{bb} against Section 3.3.2.
4. Build the measurement harness, timing each stage from the encoded bitstream inward, so that every later milestone is evaluated against Eq. 3.23 rather than against wall-clock totals.
5. Implement the budgeted controller (Eqs. 3.8–3.11) and verify budget tracking under distribution shift, before any language model is introduced.
6. Integrate the Grounder and the Reasoner’s contractual prompting; build and validate the citation check.
7. Implement two-deadline fusion with separate calibration maps; measure deadline attainment and alert stability.
8. Add the latency mechanisms one at a time: prefix cache, memoization, then speculation.
9. Add Scene Memory and run the cross-scene evaluation.
10. Run the fleet study against a simulated shared accelerator.
11. Benchmark on target edge hardware; run the full ablation set.

Milestones 4 and 5 together produce a self-contained result, a controller that holds a compute budget under distribution shift where a threshold does not. Nothing in it depends on a vision-language component working first. If the Reasoner integration proves harder than anticipated, the work still has a complete contribution rather than a partially built pipeline.

The controller, its queue coupling, the fleet reservation, the latency mechanisms and the measurement harness already exist as a simulation over generated score streams. Milestones 5, 8 and 10 therefore begin from working code. No figure from that simulation is reported as a result, since it involves no video.

Table 4.2 places the milestones on an approximate fourteen-week schedule. Detection and reasoning proceed side by side, since the Grounder and the Reasoner need nothing from the Sentinel until week nine.

TABLE 4.2: Approximate fourteen-week schedule. Milestone numbers refer to the list above.

Week	Work	Milestone
1	Data, decoding, segmentation and dataloaders. Grounder: open-vocabulary detector, tracker and the structured evidence list	6
2	Reproduce the weakly supervised baseline. Grounder validated on recorded clips	1, 6
3	Baseline compared against its published figures. Reasoner contractual prompting begins	1, 6
4	Multi-scale heads and the cross-attention merge. Reasoner prompting continues	2, 6
5	Evidential scoring head. Citation check and the re-prompt path	2, 6
6	Three-term loss, validated against the baseline. Scene Memory and the cross-scene evaluation	2, 9
7	XD-Violence, visual component, same pipeline retuned. Fleet study against a simulated shared accelerator	1–2, 10
8	Frame-cache restructuring and the measurement harness. Edge hardware: port and optimize	3–4, 11
9	Controller driven by measured (s_t, u_t) . Edge hardware: benchmark	5, 11
10	Two-deadline fusion and the two calibration maps. Human evaluation begins	7
11	Prefix cache and memoization, each measured alone. Human evaluation analysed	8

Table 4.2 (continued)

Week	Work	Milestone
12	Speculation, measured alone. First end-to-end run of the complete system	8
13	Full ablation set. Chapter 5 assembled	11
14	Final numbers and figures. Chapter 6	—

4.4 Risks and Mitigations

Three risks are worth stating in advance. If the Reasoner cannot meet a usable latency on the primary edge device even restricted to escalated segments, we report two operating points rather than one. The first is fully on-device with explanation disabled. The second is a hybrid in which the Reasoner is called over a network for escalated windows only. The two-deadline contract makes this fallback unusually cheap, since the configurations differ only in how long an amendment takes to arrive, not in whether the hard deadline is met.

If the tracker underlying the Evidence Grounder proves unreliable in crowded scenes, the evidence list degrades and the citation check verifies citations against a weaker reference. The check remains sound, since it was never claimed to validate the detections themselves, but explanation quality falls. Grounder sampling rate and tracking quality are therefore reported together, and the human evaluation of Chapter 5 is what detects this failure rather than the automatic check.

If the escalation budget proves too small for the Reasoner to observe enough of any given event, the memoization mechanism of Section 3.9.4 becomes counterproductive. It suppresses calls that would have refreshed a stale explanation. Memoization hit rate is therefore reported jointly with explanation staleness, rather than treating a high hit rate as unambiguously favourable.

Chapter 5

Experimental Results

5.1 Evaluation Protocol

A system trading accuracy for speed cannot be judged by a single number. The protocol reports four axes together for every configuration, so that no axis can be improved by silently degrading another.

5.1.1 Datasets

Two weakly supervised benchmarks anchor the comparison: UCF-Crime [44], reported with frame-level area under the ROC curve, and XD-Violence [48], reported with average precision, matching the metric conventions of the surveyed literature so numbers remain comparable. XD-Violence carries an audio track alongside its video; the video is used and the audio is not, for the reason given in Section 3.1. Results on that benchmark are therefore compared against visual-only methods, and published figures obtained with audio are marked as such rather than placed alongside ours.

Two or three camera environments are held out of training entirely and used only to evaluate cross-scene generalization, since neither primary benchmark is designed to test adaptation to an unseen camera.

For the fleet study of Section 3.4.6, we replay N streams against a single simulated accelerator with a fixed service rate.

5.1.2 Metrics

- **Detection accuracy.** Frame-level AUC and AP, reported for the Sentinel alone and for the full pipeline.
- **Deadline attainment.** Fraction of segments scored within Δ_{hard} and of escalated segments explained within Δ_{soft} , with 95th and 99th percentile latencies. Deadlines are tail properties, so means are not reported alone.
- **Budget tracking.** Realized escalation rate against the set B , plotted over time, on a dataset the controller was not tuned on.
- **Sentinel cost breakdown.** The backbone and the temporal heads timed separately, so that the approximation made in Eq. 3.23, which applies the frame cache's factor of one half to the whole of c_{sen} rather than to its backbone term alone, can be checked against the measured split.
- **Cost-model terms.** Each measurable quantity in Eq. 3.23 reported separately: ρ , h , ρ_{spec} , $\bar{\pi}$, and per-stage timings, so that predicted and observed totals can be compared. The realized escalation rate and the speculation charge $\nu\rho_{\text{spec}}(1 - \bar{\pi})$ are reported as separate quantities, since Eq. 3.24 predicts that their sum, rather than ρ alone, tracks the budget.
- **Speculation lead time.** The interval Λ_{spec} by which speculation advances the explanation, reported against the compute it wastes, so that the exchange stated in Section 3.4.4 can be judged rather than assumed favorable.
- **System cost.** Throughput and peak memory on two stated hardware tiers, measured from the encoded bitstream with decoding inside the measured region.
- **Calibration.** Expected calibration error reported separately for $\hat{p}^{\text{prov}}$ and $\hat{p}^+$.
- **Alert stability.** Amendment rate and verdict flip rate.

- **Contract compliance.** The rate at which each of the four acceptance conditions of Section 3.7 fires, reported separately, since a parse failure and an invented identifier are different faults with different remedies. The share of records recovered by the single re-prompt is reported alongside them.
- **Explanation faithfulness.** A reference-free rubric score from an independent judge model covering relevance, correctness and specificity, and a human check on a subsample of twenty to thirty clips with multiple raters, sized to be feasible rather than to constitute a large user study. The human check is what covers the failures the automatic conditions cannot detect, namely a correct citation of an incorrect detection.
- **Explanation staleness.** For explanations carried forward by memoization, the interval since the record was produced and the fraction of such alerts whose evidence had materially changed by the time they were reused, so that a high hit rate h is not read as unambiguously favorable.
- **Grounder operating point.** The sampling rate in use together with a measure of tracking quality at that rate, since Section 3.5 treats the rate as determined by tracking rather than fixed a priori, and the quality of the evidence bounds everything downstream of it.

5.1.3 Baselines and reporting discipline

Prior systems are either reproduced where code is available and compute permits, or quoted from the original publication and marked as quoted. The two are never combined into a single number. The comparison set spans the weakly supervised family [23, 44, 49] and the explanation-oriented family [27, 32, 51, 56, 58].

Every reported number carries the split, hardware and operating point it was measured under. No headline number appears without its latency and cost figures alongside. Hypotheses stated in this chapter, including the central one that a priced controller holds its escalation rate under shift where a threshold does not, are treated as claims to be confirmed or refuted, and the cost model is reported

as a predicted-versus-observed comparison in which a discrepancy is reported as a discrepancy rather than absorbed into a fitted constant.

5.1.4 Ablations

1. Sentinel alone against the full cascade.
2. Budgeted controller against a fixed threshold tuned on the source dataset, both evaluated on a different dataset and camera set, reporting realized escalation rate for each. This is the primary evidence for the controller contribution and is designed so that a null result is visible.
3. Per-frame embedding with a cache against a segment-level backbone, reporting the reduction in the backbone term c_{bb} rather than in c_{sen} as a whole, together with any change in detection accuracy, since the two arrangements differ in how spatial and temporal information are combined.
4. Latency mechanisms added individually: prefix cache, memoization, speculation, each measured for its contribution to L_{soft} and for the compute it wastes.
5. Two-deadline contract against blocking fusion, quantifying what the contract buys in tail latency and costs in alert stability.
6. Grounded explanation against an ungrounded free-text baseline receiving the same keyframes, isolating the effect of grounding from that of the underlying model.
7. Scene Memory on against off, on the held-out camera environments.
8. Learned fusion against a fixed equal-weight combination.
9. Single-scale against multi-scale Sentinel.
10. Fleet arbitration by shared price against round-robin and against per-camera fixed budgets, at equal total compute.

5.2 Results Analysis

5.3 Summary of the Experimental Results

Chapter 6

Conclusions

6.1 Research Summary

6.2 Deployment and Ethical Considerations

6.3 Future Work

Bibliography

- [1] Anusha Acharya, Hitesh Sapkota, Qi Yu, and Xumin Liu. The road less seen: Segment exploration for weakly supervised video anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14147–14156, 2026.
- [2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [3] Michèle Basseville and Igor V. Nikiforov. *Detection of Abrupt Changes: Theory and Application*. Prentice Hall, Englewood Cliffs, NJ, USA, 1993.
- [4] Daniel Bogdoll, Maximilian Nitsche, and J. Marius Zöllner. Anomaly detection in autonomous driving: A survey. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 4487–4498, 2022.
- [5] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, UK, 2004.
- [6] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter,

- Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [8] Giorgio C. Buttazzo. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Springer, 3 edition, 2011.
- [9] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4724–4733, 2017.
- [10] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.
- [11] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16901–16911, 2024.
- [12] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [13] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.
- [14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.
- [15] Zongcan Ding, Haodong Zhang, Peng Wu, Guansong Pang, Zhiwei Yang, Peng Wang, and Yanning Zhang. SlowFastVAD: Video anomaly detection via

- integrating simple detector and RAG-enhanced vision-language model. *arXiv preprint arXiv:2504.10320*, 2025.
- [16] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, pages 4878–4887, 2017.
- [17] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477): 359–378, 2007.
- [18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [19] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017.
- [20] Mahmudul Hasan, Jonghyun Choi, Jan Neumann, Amit K. Roy-Chowdhury, and Larry S. Davis. Learning temporal regularity in video sequences. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 733–742, 2016.
- [21] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [22] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [23] Hamza Karim, Keval Doshi, and Yasin Yilmaz. Real-time weakly supervised video anomaly detection. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 6848–6856, 2024.
- [24] Frank P. Kelly. Charging and rate control for elastic traffic. *European Transactions on Telecommunications*, 8(1):33–37, 1997.

- [25] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [27] Hyogun Lee, Haksub Kim, Ig-Jae Kim, and Yonghun Choi. Flashback: Memory-driven zero-shot, real-time video anomaly detection. *arXiv preprint arXiv:2505.15205*, 2025.
- [28] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 19274–19286, 2023.
- [29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [30] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 292–305. Association for Computational Linguistics, 2023.
- [31] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

- [32] Hui Lv and Qianru Sun. Video anomaly detection and explanation via large language models. *arXiv preprint arXiv:2401.05702*, 2024.
- [33] David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems*, volume 31, pages 6147–6157, 2018.
- [34] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1906–1919. Association for Computational Linguistics, 2020.
- [35] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 7076–7087, 2020.
- [36] Michael J. Neely. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.
- [37] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954.
- [38] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.
- [39] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.
- [40] Anna Rohrbach, Lisa Anne Hendricks, Kaylee Burns, Trevor Darrell, and Kate Saenko. Object hallucination in image captioning. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4035–4045. Association for Computational Linguistics, 2018.

- [41] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14318–14328. IEEE, 2022.
- [42] Murat Sensoy, Lance Kaplan, and Melih Kandemir. Evidential deep learning to quantify classification uncertainty. In *Advances in Neural Information Processing Systems*, volume 31, pages 3179–3189, 2018.
- [43] Kihyuk Sohn, David Berthelot, Chun-Liang Li, Zizhao Zhang, Nicholas Carlini, Ekin D. Cubuk, Alex Kurakin, Han Zhang, and Colin Raffel. FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *Advances in Neural Information Processing Systems*, volume 33, pages 596–608, 2020.
- [44] Waqas Sultani, Chen Chen, and Mubarak Shah. Real-world anomaly detection in surveillance videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6479–6488. IEEE, 2018.
- [45] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. Learning spatiotemporal features with 3D convolutional networks. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 4489–4497, 2015.
- [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [47] Sai Babu Veesam, Aravapalli Rama Satish, Sreenivasulu Tupakula, Yuvaraju Chinnam, Krishna Prakash, Shonak Bansal, and Mohammad Rashed Iqbal Faruque. Design of an integrated model with temporal graph attention and transformer-augmented RNNs for enhanced anomaly detection. *Scientific Reports*, 15:2692, 2025.
- [48] Peng Wu, Jing Liu, Yujia Shi, Yujia Sun, Fangtao Shao, Zhaoyang Wu, and Zhiwei Yang. Not only look, but also listen: Learning multimodal violence

- detection under weak supervision. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 322–339. Springer, 2020.
- [49] Peng Wu, Xuerong Zhou, Guansong Pang, Lingru Zhou, Qingsen Yan, Peng Wang, and Yanning Zhang. VadCLIP: Adapting vision-language models for weakly supervised video anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 6074–6082, 2024.
- [50] Tiantu Xu, Luis Materon Botelho, and Felix Xiaozhu Lin. VStore: A data store for analytics on large videos. In *Proceedings of the Fourteenth EuroSys Conference 2019*. Association for Computing Machinery, 2019.
- [51] Shengtian Yang, Yue Feng, Yingshi Liu, Jingrou Zhang, and Jie Qin. MoniTOr: Exploiting large language models with instruction for online video anomaly detection. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [52] Yuchen Yang, Kwonjoon Lee, Behzad Dariush, Yinzhi Cao, and Shao-Yuan Lo. Follow the rules: Reasoning for video anomaly detection with large language models. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 304–322, 2024.
- [53] Zhiwei Yang, Chen Gao, Jing Liu, Peng Wu, Guansong Pang, and Mike Zheng Shou. AssistPDA: An online video surveillance assistant for video anomaly prediction, detection, and analysis. *arXiv preprint arXiv:2503.21904*, 2025.
- [54] Zhiwei Yang, Chen Gao, and Mike Zheng Shou. PANDA: Towards generalist video anomaly detection via agentic AI engineer. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [55] Zhiwei Yang, Jing Liu, Guansong Pang, Peng Wu, and Zhaoyang Wu. Attention-driven pseudo-label self-training for weakly supervised video anomaly detection. *Pattern Recognition*, 177:113349, 2026.
- [56] Luca Zanella, Willi Menapace, Massimiliano Mancini, Yiming Wang, and Elisa Ricci. Harnessing large language models for training-free video anomaly

- detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18527–18536. IEEE, 2024.
- [57] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11941–11952. IEEE, 2023.
- [58] Huaxin Zhang, Xiaohao Xu, Xiang Wang, Jialong Zuo, Xiaonan Huang, Changxin Gao, Shanjun Zhang, Li Yu, and Nong Sang. Holmes-VAU: Towards long-term video anomaly understanding at any granularity. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13843–13853. IEEE, 2025.
- [59] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 1–21, 2022.
- [60] Shlomo Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83, 1996.